Acknowledgments

We would like to extend our heartfelt appreciation to our instructor, **Mrs. Beza**, whose guidance, encouragement, and commitment to academic excellence have played a significant role in the successful preparation of this Library Management System project. Her support and clear direction have been essential throughout the course.

We are also deeply grateful to our mentors, **Mr. Achenef** B. and **Mr. Getnet T**, for their consistent assistance, constructive feedback, and valuable insights during the development of this work. Their expertise and mentorship greatly enhanced the quality and depth of our project.

Our sincere thanks also go to our advisor, **Mr. Misganaw** , whose continuous supervision, thoughtful recommendations, and professional guidance were instrumental in shaping the structure and technical rigor of this documentation.

We are genuinely thankful for the combined efforts of all who contributed to our learning journey and helped us reach this milestone.

Table of contents

Chapter 1: Introduction 8

1.1 Background 8

1.2 Statement of the Problem 10

1.2.1 Cataloging and Circulation Challenges 11

1.2.2 Resource Handling and Maintenance Issues 11

1.2.3 Financial and Reporting Deficiencies 11

1.3 Objectives 11

1.3.1 General Objective: 11

1.3.2 Specific Objectives: 12

1.4 Scope 13

1.4.1 In-Scope Features 13

1.4.2 Out-of-Scope Features 14

1.5 Methodology 15

1.5.1 Overview of Agile Methodology 15

1.5.2 Detailed Process 16

1.6 Feasibility Study 17

1.6.1 Technical Feasibility 17

1.6.2. Economic Feasibility 18

1.6.3. Operational Feasibility 18

1.6.4. Legal Feasibility 18

1.6.5. Schedule Feasibility 19

Chapter 2:Requirement Elicitation 22

2.1 Overview 22

2.1.1 Elicitation Artifacts 23

2.1.2 Best Practices Applied 24

2.2 Current System and Business Process 24

2.2.1 Description of Current System 24

2.2.2 Business Process Analysis 25

2.2.3 Sub-process Details 27

2.3 Proposed System 28

2.3.1 High-Level Description 28

2.3.2 System Architecture Overview 29

2.3.3 Key Improvements Over Current System 30

2.4 Functional Requirements 31

2.4.1 Book Management (Librarian) 32

2.4.2 Member Management (Librarian) 32

2.4.3 Loan Processing (Borrowing) (Librarian) 32

2.4.4 Return Processing (Librarian) 32

2.4.5 Fine Management (Librarian) 33

2.4.6 Reporting and Analytics (Librarian) 33

2.4.7 Catalog Search (Member) 33

2.4.8 Member Account Access (Member) 33

2.4.9 Book Reservation (Member) 33

2.4.10 Book Renewal (Member) 34

2.4.11 User Management (System Administrator) 34

2.4.12 System Backup and Restore (System Administrator) 34

2.5 Non-Functional Requirement 35

2.5.1. Performance 35

2.5.2 Usability 35

2.5.3 Reliability 36

2.5.4 Security 36

2.5.5 Availability 36

2.5.6 Scalability 37

2.5.7 Maintainability 37

2.5.8 Compatibility 37

2.5.9 Data Management 37

Chapter 3: System Analysis 38

3.1 Overview 38

3.2 Use Case Diagram 38

3.2.1 System Boundary 39

3.2.2 Actors Identification 39

3.2.3 Use Cases and Detailed Descriptions 40

3.2.4 Relationships 41

3.2.5 Graphical Representation 41

3.3 Class Diagram 43

3.3.1 Overview of Classes 44

3.3.2 Detailed Class Descriptions 44

3.3.3 Relationships and Multiplicity 45

3.4 LMS Sequence Diagram 47

3.5 Activity Diagram 52

Chapter 4: System Design 57

4.1 Overview 57

4.1.1 Design Principles and Standards 58

4.1.2 Design Process Workflow 58

4.1.3 High-Level Design Choices 59

4.1.4 Integration with Previous Chapters 59

4.2 System Architecture 59

4.2.1 Architectural Style 59

4.2.2 Key Architectural Components 60

4.2.3 Data Flow Architecture 61

4.2.4 Scalability and Performance Strategies 62

4.2.5 Security Architecture 62

4.2.6 Fault Tolerance and Recovery 63

4.3 Deployment Diagram 63

4.3.1 Diagram Description 64

4.3.2 Nodes and Artifacts 66

4.3.3 Deployment Strategies 66

4.3.4 Hardware Requirements 66

4.3.5 Cloud vs. On-Premise 66

4.4 Component Diagram 66

4.4.1 Diagram Description 66

4.4.2 Key Components 67

4.4.3 Interfaces and Dependencies 68

4.4.4 Component Reuse 68

4.5 Persistent Data Model 68

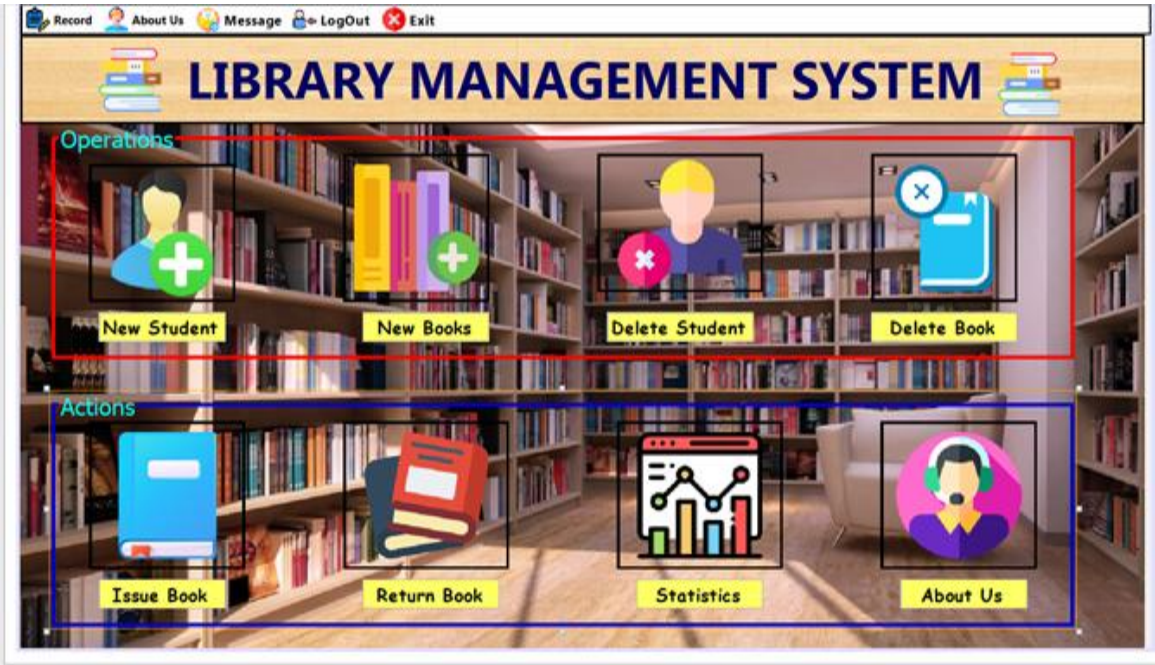
4.5.1 ER Diagram 68

4.5.2 Relational Schema 69

4.5.3 Data Access Layer 71

4.5.4 Data Integrity and Backup 71

4.5.5 Performance Optimization 71



Chapter 1: Introduction

1.1 Background

Libraries have always been fundamental pillars of education, research, and knowledge preservation. For centuries, they have served as repositories where learners, scholars, and communities access information that fuels academic progress and societal development. Despite advancements in information technologies, many libraries—particularly in educational institutions and developing regions—continue to rely on outdated, manual processes to manage their daily operations. These traditional methods often involve handwritten ledgers, paper-based borrowing cards, and physical filing systems for storing member information and tracking book circulation. While these methods were effective in earlier decades, they pose significant limitations in today's fast-paced, digital world.

The motivation for developing a Library Management System (LMS) arises from the growing complexity and scale of modern library operations. Beginning in the late 20th century, libraries slowly transitioned from purely manual systems to early computerized cataloging software. These initial systems were limited in scope, often focusing only on simple database entries for book titles and authors. They lacked integrated circulation tracking, member management, or automated notifications. As personal computers and internet access expanded in the early 2000s, libraries began adopting more comprehensive digital tools; however, many institutions still struggle

with fragmented systems that do not communicate with one another or require extensive manual intervention.

Today's academic environment demands greater efficiency, accuracy, and accessibility than traditional systems can support. The rise of digital learning platforms, online research databases, and remote access requirements has transformed user expectations. Students and faculty now anticipate real-time search results, instant access to book availability, and seamless interactions with library services. Without modern automation, libraries face challenges such as inaccurate records, misplaced materials, and time-consuming circulation tasks. A comprehensive LMS addresses these gaps by integrating catalog management, member services, circulation operations, and reporting into a unified digital platform. Such integration not only enhances operational workflows but also allows administrators to make data-driven decisions based on borrowing trends, resource usage, and collection performance.

The evolution of LMS technology reflects this growing need for modernization:

- Manual Era (Pre-1990s): Reliance on paper records for cataloging, borrowing, and member tracking resulted in slow processes and frequent errors
- Early Automation (1990s–2000s): Introduction of basic digital catalogs and standalone circulation systems with limited interaction
- Integrated Library Systems (2010s): Web-based solutions offering unified modules for cataloging, circulation, and user accounts
- Cloud-Based & Intelligent Systems (2020s–Present): Scalable platforms with analytics, remote access, mobile interfaces, and improved data security

This LMS project builds upon these technological developments to address the persistent challenges found in many library environments. It aims to replace fragmented workflows with a centralized, reliable, and user-friendly system. Statistical observations in academic institutions highlight the urgency of such modernization: studies indicate that manual record-keeping contributes to misplaced books, slow service delivery, and significant time loss for both librarians and patrons. Research also shows that automated library systems can improve circulation efficiency by 30–50% and reduce cataloging errors by over 70%, underscoring the importance of transitioning to digital solutions.

Moreover, the background of this project involves multiple stakeholders whose needs shape the system's objectives. Librarians require dependable tools for catalog maintenance, inventory tracking, and circulation management. Students and faculty expect quick access to accurate book information and seamless borrowing experiences. Administrators seek data-driven reports for decision-making, budgeting, and resource

planning. A well-designed LMS balances these needs by offering role-based access, secure data handling, and intuitive interfaces.

In summary, the background of this Library Management System reflects the historical evolution of library technologies, the practical challenges faced by institutions dependent on manual workflows, and the growing demand for efficient, scalable, and integrated solutions in modern educational environments. The system aims to bridge the gap between traditional library practices and contemporary digital capabilities, ensuring improved accuracy, user satisfaction, and operational excellence.

1.2 Statement of the Problem

Libraries in many academic institutions continue to operate using manual or semi-digital systems that are not capable of handling the increasing demands placed upon them. The absence of centralized automation affects the accuracy of book records, slows down circulation processes, and reduces the overall quality of service offered to students and staff. As the volume of library resources grows, the limitations of traditional methods—paper logs, isolated spreadsheets, and inconsistent data entry—become more apparent. These shortcomings lead to inefficiency, reduced user satisfaction, and operational bottlenecks that hinder the library's core mission of providing timely and reliable access to information.

The current manual system for managing the library is fraught with numerous challenges that hinder its efficiency and effectiveness.

Key problems include:

- Inefficiency and Time Consumption: Tasks like issuing books, returning books, and calculating fines require significant manual effort and time.
- Data Integrity and Security Issues: Physical ledgers are vulnerable to damage, loss, and unauthorized access. Data can be easily altered or become illegible.
- Difficulty in Tracking and Searching: Locating a specific book, checking its availability, or generating reports on member activity is a cumbersome and slow process.
- Lack of Real-Time Information: Patrons cannot independently check the availability of books, leading to unnecessary trips to the library and frustration.
- Error-Prone Operations: Manual data entry is susceptible to human error, which can lead to inaccurate records of inventory and member transactions.

1.2.1 Cataloging and Circulation Challenges

- ✓ Difficulty maintaining accurate and updated book records due to manual entry
- ✓ Frequent mismatches between actual book availability and recorded status
- ✓ Borrowers unable to check real-time book status
- ✓ High chance of human error during borrow/return processes
- ✓ Overdue items often go unnoticed without automated tracking
- ✓ Increased workload for librarians during peak periods

1.2.2 Resource Handling and Maintenance Issues

- ✓ No centralized system to track damaged, lost, or outdated materials
- ✓ Difficulty identifying books that require repair or replacement
- ✓ Maintenance activities recorded inconsistently in paper logs
- ✓ Delays in updating the status of books under repair or review
- ✓ Users encountering worn-out or incomplete materials
- ✓ Lack of systematic evaluation procedures for resource condition

1.2.3 Financial and Reporting Deficiencies

- ✓ Manual fine calculations leading to mistakes and inconsistencies
- ✓ Fee records vulnerable to loss or unauthorized changes
- ✓ Lack of transparency in financial transactions
- ✓ Time-consuming and error-prone report generation
- ✓ Difficulty producing accurate data for budgeting decisions
- ✓ Limited ability to analyze borrowing trends or resource usage

These problems collectively result in operational inefficiencies, poor user satisfaction, and an increased workload for library staff.

1.3 Objectives

The project's objectives are divided into General and Specific to provide clear direction.

1.3.1 General Objective:

The primary goal of this project is to design, develop, and implement a robust, user-friendly, and automated Library Management System that streamlines all major library operations, thereby increasing efficiency, reducing errors, and significantly improving the service quality for both librarians and patrons.

1.3.2 Specific Objectives:

- ✓ Catalog Management: Develop a centralized module for adding, updating, and organizing book records with real-time availability updates to reduce catalog inconsistencies.
- ✓ Circulation Operations: Enable fast and accurate book borrowing, returning, and renewal processes through automated validation and due-date generation.
- ✓ Member Services: Provide digital registration, profile management, and personalized account dashboards to improve user accessibility and reduce administrative delays.
- ✓ Fine and Fee Automation: Implement automatic fine calculation, record keeping, and payment tracking to eliminate manual errors and improve financial transparency.
- ✓ Resource Monitoring: Create tools to track lost, damaged, and under-maintenance books with real-time status updates for better collection management.
- ✓ Search and Discovery: Deliver advanced search capabilities using title, author, ISBN, and category filters to improve resource discovery and reduce search times.
- ✓ Reporting and Analytics: Generate customizable reports on circulation trends, overdue items, inventory statistics, and user activity for informed decision-making.
- ✓ Security Enhancements: Incorporate role-based access control, data encryption, and audit logging to ensure data protection and compliance with institutional policies.
- ✓ User Interface Design: Ensure intuitive, responsive, and accessible web interfaces that support both library staff and student users across devices.
- ✓ Scalability and Integration: Support growth in books, members, and transactions with an architecture prepared for future integration such as barcode scanners or digital archives.

These objectives align with modern library automation standards and aim to increase efficiency, reduce errors, and enhance both staff productivity and user satisfaction.

1.4 Scope

The scope defines the boundaries of the Library Management System (LMS) for this development phase. It clearly outlines the features, functionalities, and user roles that will be included, as well as those explicitly excluded to ensure a focused and manageable project.

1.4.1 In-Scope Features

The following modules and functionalities will be developed and implemented in this version of the system:

1. **User Management Module:**
    - ✓ **Librarian/Admin Role:** Full system access, including the ability to add, modify, and deactivate member accounts; manage all book records; and process all transactions.
    - ✓ **Member/Student Role:** Ability to view their own profile, search the book catalog, and see their personal borrowing history and current loans.
2. **Book/Catalog Management Module:**
    - ✓ **CRUD Operations:** The Librarian can Create, Read, Update, and Delete (CRUD) book records in the digital catalog.
    - ✓ **Book Information:** Each record will store key details such as Book ID, Title, Author(s), ISBN, Publisher, Publication Year, Genre/Category, and Number of Copies.
    - ✓ **Book Status:** Automatic tracking of a book's status (e.g., Available, Checked-Out, Lost).
3. **Transaction Management Module:**
    - ✓ **Book Borrowing (Check-out):** Functionality for a librarian to issue a book to a valid member, linking the book to the member's account and setting a due date.
    - ✓ **Book Returning (Check-in):** Functionality for a librarian to receive a returned book, update its status to "Available," and clear it from the member's account.

- ✓ **Loan Period & Validation:** Enforcement of a standard loan period (e.g., 14 days) and validation rules (e.g., a member cannot borrow if they have overdue books or have reached the maximum loan limit).

4. **Search and Discovery Module:**
    - ✓ **Basic Search:** A simple, functional search bar allowing users (both librarians and members) to search for books by **Title, Author, or ISBN**.
5. **Fine Management Module:**
    - ✓ **Automatic Fine Calculation:** The system will automatically calculate overdue fines based on a fixed, pre-defined daily rate when a book is returned after its due date.
    - ✓ **Fine Tracking:** Fines will be recorded against the member's account.
6. **Reporting Module (Basic):**
    - ✓ Generation of essential reports, including:
        - Inventory Report (List of all books).
        - List of Currently Borrowed Books.
        - Overdue Books Report.
        - Member Transaction History

1.4.2 Out-of-Scope Features

The following functionalities are **explicitly excluded** from the current project plan but may be considered for future versions:

- ➢ Advanced Reservations: Users cannot place holds or reserve books that are currently checked out.
- ➢ Digital Content Management: The system will not handle e-books, audiobooks, or other digital media.
- ➢ Financial Transactions: Integration with payment gateways for online fine payments will not be implemented. Fines will be tracked but paid offline.
- ➢ Hardware Integration: The system will not integrate with barcode scanners, RFID readers, or receipt printers.

- Multi-Branch Support: The system is designed for a single library branch.
- Public Web Portal / OPAC: A separate online public access catalog for unauthenticated users to search the catalog without logging in will not be developed.
- Advanced Notifications: Automated email or SMS reminders for due dates will not be included (though this may be a manual process).
- Advanced Analytics: Predictive analytics for book popularity or user behavior is beyond the scope.
- Inter-Library Loan Management: Functionality to manage loans between different libraries is excluded.

1.5 Methodology

1.5.1 Overview of Agile Methodology

This project will be developed using the Agile (Scrum) Methodology, following an object-oriented approach with iterative and incremental development.

Reason for Choice: Agile is ideal for the Digital Library Management System because:

- ✓ Requirements evolve — librarians, students, and staff may request new features (e.g., mobile access, advanced search filters) during development.
- ✓ Early feedback is critical — delivering working modules (e.g., book search, borrowing) allows real users to test and refine functionality.
- ✓ Team collaboration — daily stand-ups and sprint reviews foster communication among the 6-member team.
- ✓ Risk reduction — frequent testing and integration prevent major issues at the end.
- ✓ Flexibility — changes can be accommodated without restarting the project.

The **object-oriented paradigm** complements Agile by modeling real-world entities (Book, Member, Borrowing) as reusable, modular classes, improving code maintainability and scalability.

1.5.2 Detailed Process

The project will be divided into five 2-week sprints, each delivering a tested, working increment of the system. Each sprint follows the Scrum cycle: Sprint Planning → Daily Stand-ups → Development → Sprint Review → Retrospective.

Sprint	Duration	Goals & Deliverables
Sprint 1	Week 1–3	**Core Foundation** • User authentication (Student, Staff, Librarian, Admin) • Database setup (MySQL) • Book catalog management (Add/Edit/Delete)
Sprint 2	Week 4–6	**Borrowing & Return Module** • Issue/return books with barcode scan • Due date calculation • Overdue tracking
Sprint 3	Week 7–10	**Search & Reservation System** • Full-text search by title, author, ISBN • Online reservation & waitlist • Email/SMS notifications
Sprint 4	Week 11–13	**Reporting & Admin Dashboard** • Usage analytics (most borrowed, overdue reports) • Fine management (0.50 ETB/day) • System logs
Sprint 5	Week 14–16	**Final Integration & Deployment** • UI/UX polish (responsive design) • Comprehensive testing (unit, integration, user

Sprint	Duration	Goals & Deliverables
		acceptance) • Deployment on UoG server • User training guide

Tools and Technologies:

Category	Tool/Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	User interface
Backend	PHP (or Node.js), REST APIs	Server-side logic
Database	MySQL/Oracle/PostgreSQL	Resource management
Hosting	University Server, AWS/Netlify	Deployment
Version Control	Git, GitHub	Collaboration
Modeling	UML Diagrams	Design & Architecture

1.6 Feasibility Study

A feasibility study assesses the project's viability from multiple perspectives:

1.6.1 Technical Feasibility

- Technology Stack: The system will use established technologies (Java/Python, MySQL/PostgreSQL, HTML/CSS/JavaScript) that are well-documented and within the team's capability
- Infrastructure Requirements: Standard web servers and database systems are sufficient

- ❖ Integration: No complex hardware integration required for initial version
- ❖ Conclusion: Technically feasible with available resources and expertise

1.6.2. Economic Feasibility

- ✓ Development Costs: Primarily development time (no licensing costs for open-source tools)
- ✓ Hardware Costs: Minimal (uses existing computer systems)
- ✓ Operational Savings:
  - Reduced staff time on manual processes
  - Decreased book loss through better tracking
  - Improved resource utilization
- ✓ Return on Investment: Positive ROI expected within first year of operation
- ✓ Conclusion: Economically justified with low costs and significant long-term benefits

1.6.3. Operational Feasibility

- User Acceptance: High - addresses clear pain points of current manual system
- Training Requirements: Minimal - intuitive web interface
- Process Impact: Streamlines existing workflows without major disruption
- Staff Capability: Library staff possess basic computer skills required
- Conclusion: Operationally feasible with smooth transition expected

1.6.4. Legal Feasibility

- ➢ Data Privacy: Compliance with data protection regulations for member information
- ➢ Licensing: All software components are open-source or properly licensed
- ➢ Copyright: System designed to manage physical books without digital rights issues
- ➢ Conclusion**:** No significant legal barriers identified

1.6.5. Schedule Feasibility

- ✓ Timeframe: 16-week development schedule using agile approach
- ✓ Milestones: Clear deliverables for each iteration
- ✓ Resources: Adequate team size and skills available
- ✓ Risk Management: agile approach mitigates schedule risks
- ✓ Conclusion: Schedule is realistic and achievable

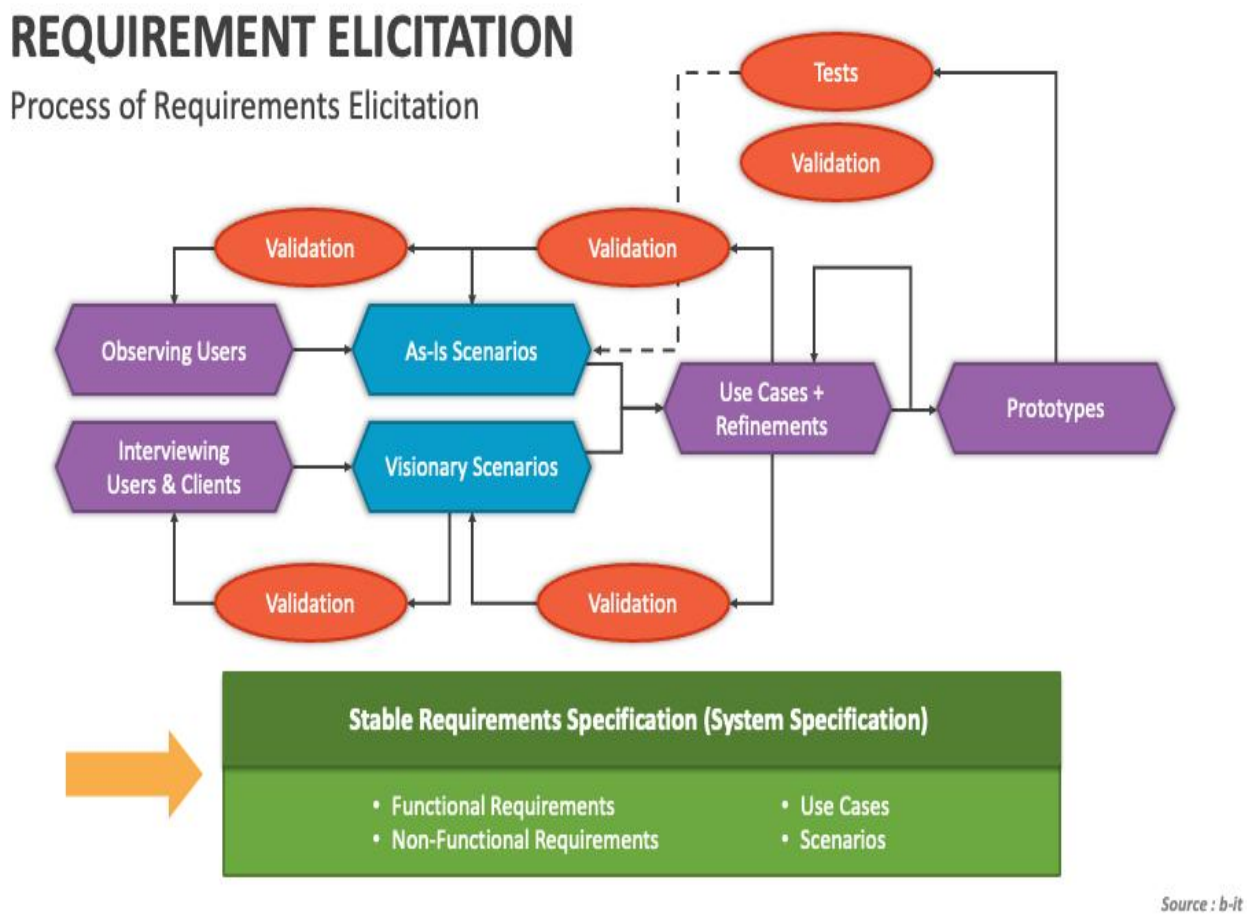
Overall Conclusion: The Library Management System project is feasible across all dimensions and recommended for implementation.

Feasibility Type	Description	Assessment	Conclusion
Technical Feasibility	Evaluates whether the required technology, tools, and team skills are available to build the system.	• **Technology**: PHP, MySQL, HTML/CSS/JS (widely available and free). • **Hardware**: Existing UoG server (8GB RAM, 4-core CPU) is sufficient. • **Team Skills**: 6 members proficient in web development and OOP. • **Tools**: Apache, Git, VS Code, Postman.	**Feasible**
Schedule Feasibility	Assesses whether the project can be completed within the given timeline.	• **Duration**: 16 weeks (4 sprints × 4 weeks). • **Team Size**: 6 members (2 frontend, 2 backend, 1 DB, 1 tester). • **Buffer**: 2 week for delays.	**Feasible**

Feasibility Type	Description	Assessment	Conclusion
		• **Milestones**: MVP in 8 weeks, full system in 16 weeks.	
Legal and Ethical Feasibility	Ensures compliance with laws, university policies, and ethical standards.	• **Data Privacy**: User data encrypted (HTTPS, password hashing). • **Copyright**: Only metadata stored (no full-text e-books). • **UoG IT Policy**: System runs on internal network. • **Licensing**: Open-source tools (MIT/BSD). • **Ethics**: Transparent fine system, audit logs.	**Feasible**
Operational Feasibility	Checks if the system will be accepted and used effectively by end-users.	• **User Training**: Simple UI + manual provided. • **Stakeholder Buy-in**: Librarians involved in requirements. • **Change Management**: Phased rollout. • **Support**: Admin panel for issue tracking.	**Feasible**
Economic Feasibility	Analyzes cost-benefit to justify development.	• **Development Cost**: $0 (team labor + open-source tools). • **Hardware**: Uses existing server. • **Benefits**: Saves 100+ staff hours/month, reduces lost books,	**Highly Feasible**

Feasibility Type	Description	Assessment	Conclusion
		improves user satisfaction. • **ROI**: Immediate (no licensing fees).	

Chapter 2:Requirement Elicitation



2.1 Overview

This chapter documents the comprehensive process of gathering, analyzing, and specifying the requirements for the proposed **Library Management System (LMS).** The primary objective is to bridge the gap between the identified problems of the existing manual operations and the capabilities of a new, automated system. This is achieved by thoroughly understanding the needs and expectations of all key stakeholders, including library staff, patrons, and system administrators.

Requirement elicitation is a critical phase in the software development life cycle (SDLC) where stakeholders' needs, expectations, and constraints are gathered,

analyzed, and documented. For the Library Management System (LMS) project, this chapter focuses on systematically identifying requirements to ensure the proposed system aligns with library operations, user needs, and administrative goals.

Elicitation techniques include interviews with librarians, questionnaires for frequent patrons, and direct observation of workflows. The objective is to bridge the gap between the limitations of the current manual system and a fully integrated digital library platform.

Key principles guiding this phase include:

- ✓ Stakeholder Involvement: Engaging librarians, administrative staff, and library members to gather comprehensive inputs
- ✓ Iterative Approach: Refining requirements through multiple feedback cycles
- ✓ Traceability: Ensuring each requirement can be traced to stakeholder needs and design elements
- ✓ Prioritization: Categorizing requirements as must-have, should-have, could-have, won't-have to manage scope

Techniques employed in this LMS project:

- ✓ Observation: Shadowing library operations to identify inefficiencies like manual ledger updates and time-consuming book check-outs
- ✓ Workshops: Group sessions with staff to ideate features like automated borrowing and reporting
- ✓ Document Analysis: Reviewing existing records, ledgers, and spreadsheets to establish a baseline

Challenges included conflicting requirements (e.g., librarians needing detailed reports vs. members desiring simple self-service) and assumptions about digital literacy. These were resolved via prioritization matrices and validation sessions.

Requirements are categorized into **Functional** (what the system does) and **Non-Functional** (how it performs: security, usability, reliability), forming the foundation for system design.

2.1.1 Elicitation Artifacts

Artifacts include requirement traceability matrices (RTM), user personas (e.g., "Library Clerk: 25–35 years old, manages ~50 transactions daily"), and context diagrams showing system boundaries.

2.1.2 Best Practices Applied

Adhered to IEEE Std 830-1998 for requirement specifications. Tools such as Jira for tracking and StarUML for preliminary diagrams were used.

This detailed overview ensures a solid elicitation foundation, contributing to project success.

2.2 Current System and Business Process

The current system in the library relies predominantly on manual paperbased processes and fragmented digital tools like basic spreadsheets leading to significant operational inefficiencies This section analyzes the existing setup core business processes and associated challenges to clearly justify the need for the proposed Library Management System LMS

2.2.1 Description of Current System

The library's operations are managed through a combination of physical ledgers index cards and standalone software with no integration.

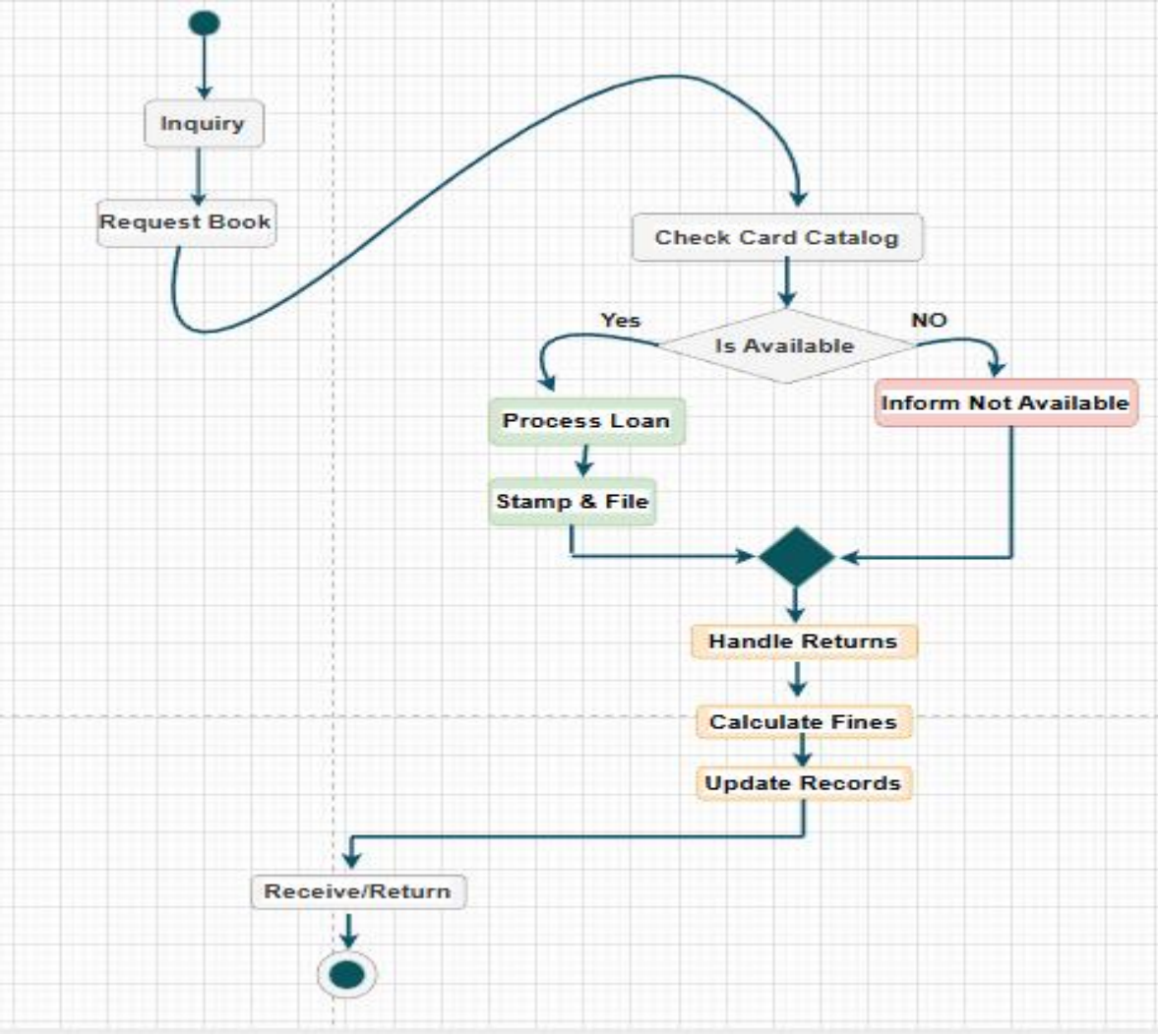
Key components include:

- ✓ Catalog Management: A physical card catalog system for searching books supplemented by a nonnetworked Excel spreadsheet for new acquisitions The spreadsheet and card catalog often have discrepancies
- ✓ Member Registration: Paperbased registration forms stored in filing cabinets Member IDs are manually issued and there is no centralized database of members
- ✓ Circulation Operations: A manual checkout process using paper slips stored in a duedate rack Returns involve manually sorting these slips and calculating fines
- ✓ Inventory Management: Annual stocktaking is performed by physically checking every shelf against the card catalog or spreadsheet a process that takes several days
- ✓ Fine Management: Fines are recorded in a cash ledger and calculated manually leading to inconsistencies
- ✓ Reporting: Reports on overdue books or popular titles are compiled adhoc by manually sifting through circulation slips and ledgers

Hardware typically includes a few standalone desktop computers. Software is limited to basic Microsoft Office applications, with no centralized database or network connectivity between workstations.

2.2.2 Business Process Analysis

The library's core business processes are sequential laborintensive and prone to errors due to their reliance on manual handling and nonintegrated records.



Process 1 Member Registration

- ✓ A prospective member fills out a paper registration form

- ✓ A librarian manually verifies the information student ID
- ✓ The librarian creates a physical library card by writing the member's details
- ✓ The paper form is filed alphabetically in a cabinet
- ✓ The member is given the physical card

Challenges Process takes 57 minutes per member Forms can be lost or misfiled Duplicate registrations are possible with no system to check

Process 2 Book Borrowing Checkout

1. Member presents the book s and library card
2. Librarian retrieves the book's card from the pocket inside the book
3. Librarian stamps the due date on both the book's card and a separate circulation slip
4. The book's card is filed in a circulation tray under the member's name or due date
5. The circulation slip is filed in a separate duedate rack

Challenges Timeconsuming 35 minutes per transaction High risk of error in date stamping or filing No realtime visibility on a member's current loans or status

Process 3 Book Returning Checkin

- ➢ Member returns the book to the counter
- ➢ Librarian retrieves the corresponding circulation slip from the duedate rack
- ➢ Librarian locates and retrieves the book's card from the circulation tray
- ➢ The book's card is placed back inside the book
- ➢ The book is shelved manually
  If overdue the librarian manually calculates the fine using a calendar and records it in a ledger

Challenges Slow and inefficient 46 minutes per return Difficult to manage if a slip is missing Manual fine calculation is errorprone and can lead to disputes

Process 4 Inventory StockTaking

- ➢ Library is closed for 23 days for stocktaking
- ➢ Staff members are assigned sections of shelves
- ➢ Each staff member physically checks each book on the shelf against the card catalog or Excel spreadsheet

- Discrepancies for example book on shelf but not in catalog or viceversa are noted on a sheet
- Sheets are compiled and the master catalog spreadsheet is manually updated

Challenges Extremely laborintensive and requires closing the library High probability of human error during the checking and updating process The inventory record is only accurate on the day of the count

2.2.3 Sub-process Details

A**. Sub-process:** Manual Fine Calculation and Collection within Process 3

- **Activities:**

- ✓ Retrieve return slip and note the due date
- ✓ Check the current date
- ✓ Calculate the number of overdue days manually using a physical calendar
- ✓ Multiply days by the daily fine rate
- ✓ Collect cash payment
- ✓ Record the payment in a cash ledger
- ✓ Issue a handwritten receipt

- Actors: Librarian Member:
- Inputs: Returned book Due Date Current Date Fine Rate Policy
- Outputs: Cash Payment Ledger Entry Handwritten Receipt
- Pain Points: Calculation errors lost receipts difficulty tracking payment history risk of unauthorized waivers and insecure cash handling

B. **Sub-process:** Manual Search for Book Availability within a Patron's Request

- **Activities:**

- ✓ Patron requests a book by title or author
- ✓ Librarian searches the physical card catalog or the Excel spreadsheet on a local computer
- ✓ If found in the catalog the librarian checks the circulation tray to see if the book's card is present indicating it is checked out or absent indicating it should be on the shelf
- ✓ Librarian may need to physically check the shelf if the card is missing

- Actors Librarian Patron

- Inputs Book Title Author
- Outputs Verbal confirmation of availability and location
- Pain Points Slow response time 24 minutes inaccurate status if cards are misfiled inability for patrons to perform searches independently and librarian time consumed by simple queries

2.3 Proposed System

The proposed system fundamentally redefines how the library functions by replacing scattered manual tasks with a coordinated digital platform. The intent is strong, but a plan like this must be examined beyond the ideal goals to expose weaknesses, assumptions, and potential blind spots. Below is a breakdown with a more critical stance while keeping your list format free of any dots.

2.3.1 High-Level Description

The LMS is designed as a webresponsive system supported by a relational database and modular architecture. The idea is appropriate for a modern library, but several hidden complexities should be acknowledged.

Core Components
• Database A centralized relational database such as MySQL or PostgreSQL responsible for handling book records patrons transactions and fines
• Frontend A responsive interface built with modern web technologies for use on multiple devices
• Backend A structured framework such as NodeJS or Django for business rules and data logic
• Integrations Future additions such as email notifications or online payments

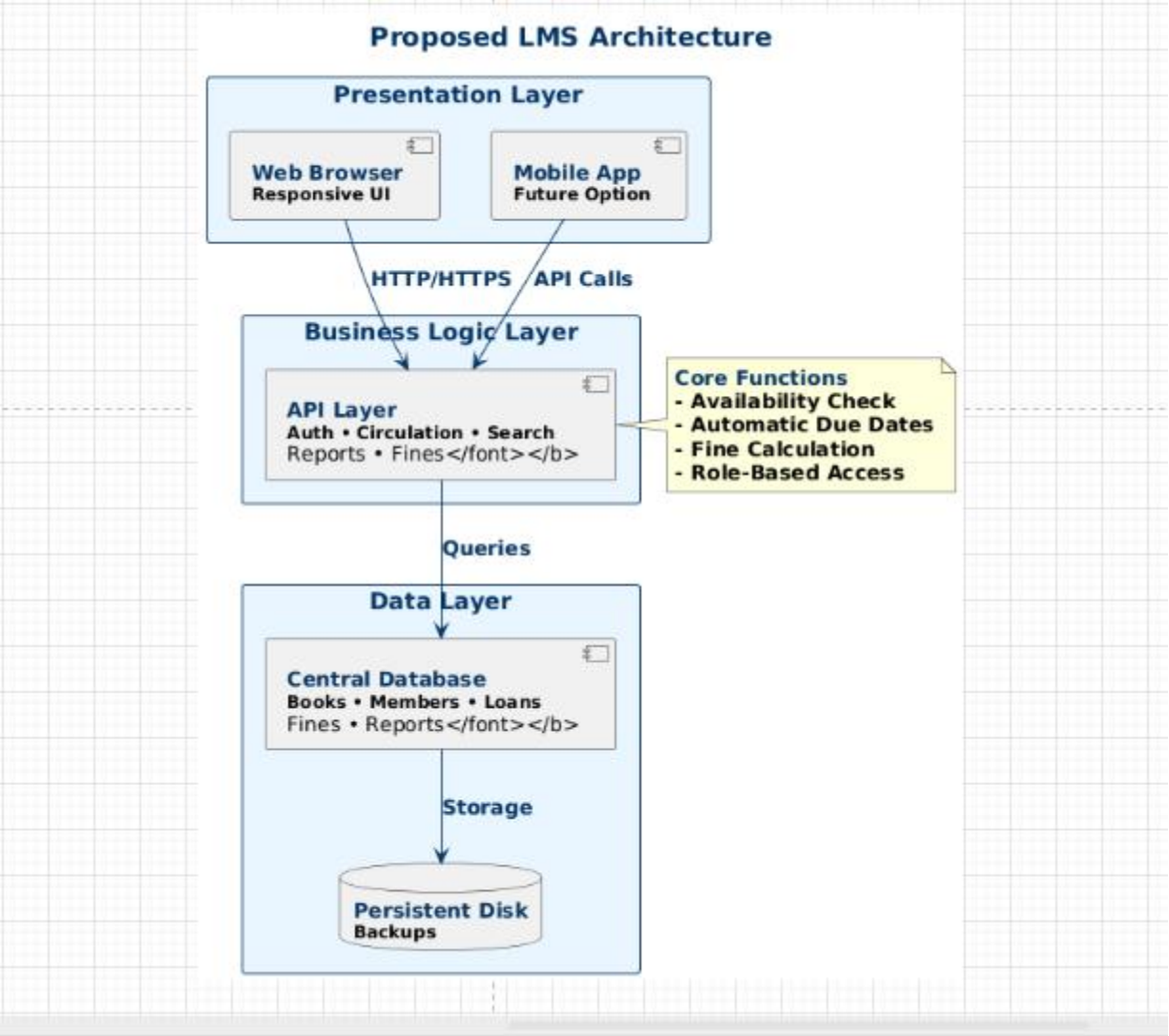
Critical Perspective
Your design assumes that all patrons and librarians will consistently have access to modern devices and stable connectivity. In practice
– Some users may rely on older hardware
– Network issues can degrade experience
– Training needs might be underestimated

You might want to challenge whether a fully webbased approach is ideal without providing offline fallback mechanisms.

2.3.2 System Architecture Overview

The system follows a three-tier clientserver model separating presentation logic and data. This is solid and scalable but not without challenges.

Critical Considerations
– Three-tier systems require disciplined deployment and version control
– Hosting on a university server depends on institutional technical capacity
– Cloud hosting introduces recurring costs and security responsibilities
– A lot of libraries underestimate the complexity of maintaining rolebased access plus backups
– You assume a stable hosting environment but many institutions struggle with server maintenance



If infrastructure is weak you may need hybrid deployment options or local caching approaches

2.3.3 Key Improvements Over Current System

The proposed improvements directly address major weaknesses in the manual system but each improvement also introduces operational risks that must be considered

- Centralized Digital Catalog
  - Removes scattered paper and spreadsheet records
  - Ensures consistency and reduces duplication
  - Challenge Requires careful migration of legacy data which is often messy incomplete or unreadable

- Automated Circulation
  • Provides automatic due dates fine calculation and real time status updates
  • Challenge Needs precise rules and strong exception handling for cases such as damaged items or manual overrides
- Online Public Access Catalog OPAC
  • Enables independent searching and access to account details
  • Challenge Depends heavily on user digital literacy which varies widely among patrons
- SelfService Portal
  • Allows online renewals reservations and account management
  • Challenge Risk of unfair access or reservation conflicts if rules are not clearly defined and enforced
- Enhanced Reporting Dashboard
  • Generates automatic statistics insights and circulation trends
  • Challenge Accuracy fully depends on staff data entry discipline and consistent system usage
- RoleBased Security
  • Restricts sensitive operations to authorized librarians only
  • Challenge Misconfigured access rules can expose sensitive data or prevent staff from performing essential tasks
- Assumptions
  • Reliable internet connectivity
  • Basic digital literacy among users
  • Training will be provided
  • Challenge These assumptions often fail in real environments leading to slow adoption or user resistance

2.4 Functional Requirements

Functional Requirements (FRs) define the specific behaviors, functions, and services that the system must provide. They describe what the system should do, without specifying the implementation details. The following requirements are categorized by the primary actor who initiates the interaction.

Functional requirements describe the essential capabilities the LMS must provide. They are based on the needs of librarians, members, and administrators and are organized by core functions.

2.4.1 Book Management (Librarian)

FR1.1 The system shall allow authorized librarians to add new books with full bibliographic details.
FR1.2 The system shall allow editing of existing book information including quantity, edition, and status.
FR1.3 The system shall support deactivation of books that are lost, damaged, or removed from circulation.
FR1.4 The system shall provide advanced search and filtering options for locating book records.
Priority: Must-have

2.4.2 Member Management (Librarian)

FR2.1 The system shall support registration of new members by capturing personal details.
FR2.2 The system shall allow updating of existing member profiles.
FR2.3 The system shall allow blocking or deactivating members who violate borrowing policies.
Priority: Must-have

2.4.3 Loan Processing (Borrowing) (Librarian)

FR3.1 The system shall issue a loan when a valid member ID and book ID are entered or scanned.
FR3.2 The system shall automatically record loan details including issue date and due date.
FR3.3 The system shall prevent borrowing if limits are exceeded, fines are high, or book availability is zero.
Priority: Must-have

2.4.4 Return Processing (Librarian)

FR4.1 The system shall record a book return when a book ID is entered or scanned.
FR4.2 The system shall automatically calculate overdue fines based on return date and policy.
FR4.3 The system shall update the book status to available and refresh the member's borrowing record.
Priority: Must-have

2.4.5 Fine Management (Librarian)

FR5.1 The system shall display fines associated with a member and their borrowed books.
FR5.2 The system shall allow recording of fine payments and updating of balances.
FR5.3 The system shall allow librarians to waive fines with a required justification.
Priority: High

2.4.6 Reporting and Analytics (Librarian)

FR6.1 The system shall generate reports on overdue books.
FR6.2 The system shall provide borrowing history for books and members.
FR6.3 The system shall provide analytics such as frequently borrowed titles and active membership.
Priority: Medium

2.4.7 Catalog Search (Member)

FR7.1 The system shall allow public search by Title, Author, ISBN, Subject, or Keyword without login.
FR7.2 The system shall display book details with real-time availability status.
Priority: Must-have

2.4.8 Member Account Access (Member)

FR8.1 The system shall display borrowed books and due dates on the user dashboard.
FR8.2 The system shall show the member's borrowing history.
FR8.3 The system shall display outstanding fines.
Priority: Must-have

2.4.9 Book Reservation (Member)

FR9.1 The system shall allow members to reserve books currently on loan.
FR9.2 The system shall notify the member automatically when the reserved book is returned.
FR9.3 The system shall prevent renewals on books that have active reservations.
Priority: High

2.4.10 Book Renewal (Member)

FR10.1 The system shall allow members to renew borrowed books if no reservation exists.
FR10.2 The system shall automatically assign a new due date upon successful renewal.
Priority: Medium

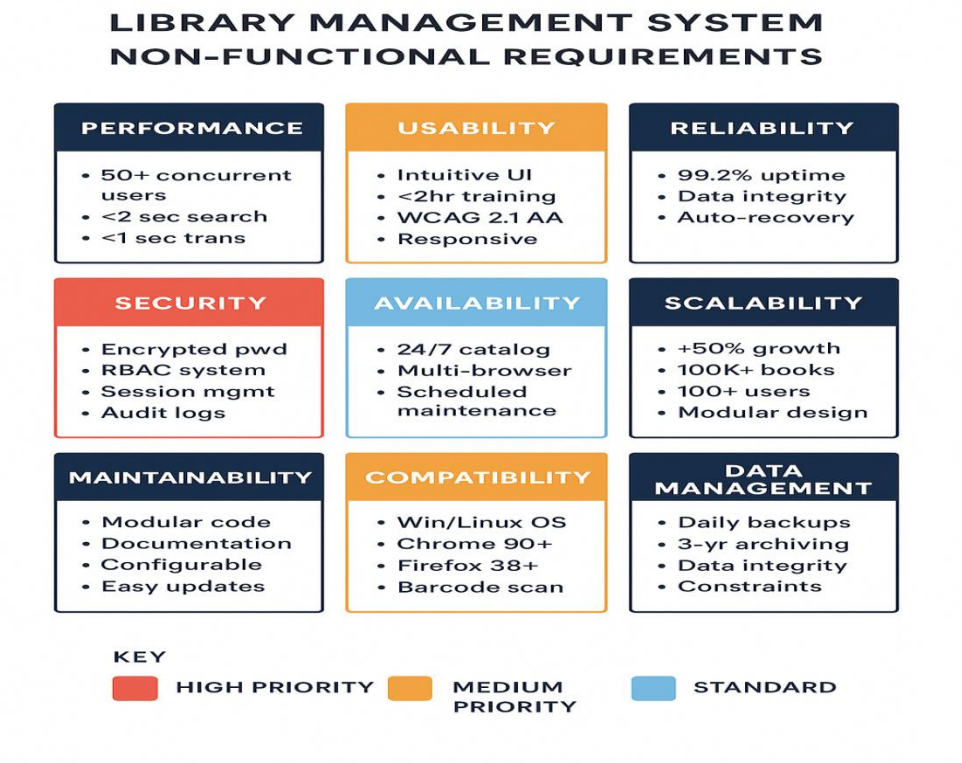
2.4.11 User Management (System Administrator)

FR11.1 The system shall allow creation of librarian accounts.
FR11.2 The system shall allow assignment of roles and permissions.
FR11.3 The system shall support deactivation of user accounts.
Priority: Must-have

2.4.12 System Backup and Restore (System Administrator)

FR12.1 The system shall allow manual initiation of full database backups.
FR12.2 The system shall allow viewing and managing previous backup records.
FR12.3 The system shall support restoration of data from existing backups.
Priority: High

2.5 Non-Functional Requirement



Non-Functional Requirements define the quality attributes, constraints, and overall performance criteria for the Library Management System. They specify *how well* the system must perform, rather than *what* it does.

2.5.1. Performance

- The system should support at least **50 concurrent users** (librarians and members) without significant performance degradation.
- **Search operations** in the catalog should return results in **less than 2 seconds**.
- **Critical transactions** (e.g., book checkout, return) should be processed in **under 1 second**.
- The system should have an initial application load time of under 5 seconds.

2.5.2 Usability

- ✓ The user interface shall be **intuitive and easy to use** for both librarians and members with minimal training.

- ✓ The system shall follow **consistent design principles** across all pages (navigation, buttons, forms, terminology).
- ✓ The system should provide **context-sensitive help** and tooltips for major functions.
- ✓ The interface should be **responsive** and work effectively on different devices and screen sizes.

2.5.3 Reliability

- ➢ The system shall have an **uptime of 99.2%** during library operating hours.
- ➢ Data corruption due to software failure shall be prevented through **database transaction management**.
- ➢ The system should be able to **recover automatically** from minor software faults without administrator intervention.
- ➢ All data entered by users shall be validated to prevent incorrect information from being stored.

2.5.4 Security

- All user passwords shall be **encrypted** in the database using strong hashing algorithms.
- **Role-based access control** should be implemented to restrict functionality based on user roles.
- All user sessions shall expire after **15-30 minutes of inactivity**.
- All sensitive data transmission shall be encrypted using **HTTPS/TLS** protocols.
- The system shall maintain **audit logs** of all significant activities (logins, transactions, data changes).

2.5.5 Availability

- ✓ The web-based system shall be accessible from any **standard web browser** (Chrome, Firefox, Safari, Edge) within the library network.
- ✓ If policy allows, the system should be accessible from the **public internet** with proper security measures.
- ✓ Scheduled maintenance windows shall be announced at least **24 hours in advance**.

2.5.6 Scalability

- ✓ The system architecture shall be capable of handling a **50% increase** in books, members, and transactions over the next 5 years.
- ✓ The database design should support expansion to **100,000+ book records** without performance degradation.
- ✓ The system should maintain performance when the number of concurrent users increases to **100+**.

2.5.7 Maintainability

- ❖ The system should be designed with **modular components** that can be updated independently.
- ❖ The source code shall be **well-documented** and follow standard coding conventions.
- ❖ System configuration settings shall be easily modifiable without requiring code changes.
- ❖ Database schema changes should be manageable through version-controlled migration scripts.

2.5.8 Compatibility

- ❖ The system shall be compatible with **Windows 10/11** and **major Linux distributions** for server deployment.
- ❖ The client interface shall work on **Chrome 90+**, **Firefox 88+**, **Safari 14+**, and **Edge 90+**.
- ❖ The system should be integrated with standard **bar code scanners** and **receipt printers**.

2.5.9 Data Management

- ➢ The system shall automatically perform **daily backups** of all critical data.
- ➢ Historical transaction data shall be archival after 3 years to maintain system performance.
- ➢ The database shall enforce **data integrity constraints** to prevent orphaned records and inconsistent data.

Chapter 3: System Analysis

3.1 Overview

This chapter transforms the functional and non-functional requirements identified in Chapter 2 into precise visual and structural models of the Library Management System (LMS). System analysis serves as the crucial bridge between what the system should do (requirements) and how it will be implemented (design). Using standardized Unified Modeling Language (UML) diagrams, we define the system's structure, behavior, and interactions. This analysis ensures a clear, unambiguous understanding of the system's functionality among all stakeholders—developers, clients, and end-users—before moving into the design and development phases.

The primary goal of this phase is to build a conceptual model that clarifies:

- ✓ The system's key entities and their relationships.
- ✓ How different users (actors) interact with the system to achieve their goals.
- ✓ The flow of events for critical business processes.
- ✓ The dynamic behavior of the system in response to specific operations.

The main outputs of this chapter include:

- ➢ Use Case Diagram (Provided): A high-level overview of system functionality and actors.
- ➢ Class Diagram: A static structure showing the system's entities, their attributes, and relationships.
- ➢ Sequence Diagram: A dynamic model illustrating the interaction between objects over time for specific use cases.
- ➢ Activity Diagram: A flowchart representing the procedural flow of control between activities for a business process.

3.2 Use Case Diagram

A Use Case Diagram is a visual representation of a system's functional requirements from the perspective of its end-users. It captures the interactions between the system's external actors (users or other systems) and the use cases (functionalities) that the

system provides. This diagram serves as a high-level overview of the system's behavior, helping to define the project's scope and clarify the roles of different users.

3.2.1 System Boundary

The system boundary is represented by a rectangle that encloses all the use cases of the Library Management System. This clearly distinguishes the system's internal functionalities from the external actors who interact with it.

3.2.2 Actors Identification

Actors are roles played by external entities that interact with the system. The primary actors for this Library Management System are:

1. **Librarian:**
    - **Description:** The primary user responsible for the day-to-day operational management of the library. This actor has the most comprehensive access to the system.
    - **Characteristics:** Typically a library staff member with administrative privileges over library resources and member transactions.
2. **Member/Patron:**
    - **Description:** A registered user of the library who borrows books and uses library services. This actor interacts with a limited set of self-service features.
    - **Characteristics:** Typically a student or faculty member. They can view their own data but cannot access other members' information or administrative functions.
3. **System Administrator:**
    - **Description:** Responsible for the technical maintenance, configuration, and user management of the system itself. This role is focused on the system's health and security rather than daily library operations.
    - **Characteristics:** Has technical expertise and highest-level system access, typically an IT staff member.

3.2.3 Use Cases and Detailed Descriptions

Use Case Name	Primary Actor	Description
Manage Books	Librarian	Allows the librarian to add new books to the catalog, update existing book information (e.g., title, author, copies), and delete or deactivate book records.
Manage Members	Librarian	Enables the librarian to register new library members, update their profile information (phone, email), and deactivate member accounts if necessary.
Process Book Loan	Librarian	Facilitates the book borrowing process by issuing a book to a valid member, recording the transaction, and automatically setting a due date.
Process Book Return	Librarian	Facilitates the book return process by recording the return, updating the book's status to "Available," and calculating any applicable overdue fines.
Manage Fines	Librarian	Allows the librarian to view fine records, record payments made against fines, and waive fines with a mandatory reason.
Generate Reports	Librarian	Enables generation of operational reports on demand, including Overdue Books, Borrowing History, Popular Books, and Member Lists.
Search Catalog	Member / Patron	Allows members to search the library's catalog by Title, Author, ISBN, or Keyword without logging in. Displays real-time availability.
View Account	Member / Patron	Enables a logged-in member to view their personal dashboard, including currently borrowed books, due dates, loan history, and outstanding fines.
Reserve Book	Member / Patron	Allows a member to place a reservation on a book currently checked out by another user. The system holds the book for them upon its return.
Renew Book	Member / Patron	Allows a member to request an extension for a currently borrowed book, subject to validation rules such as no active reservations by others.

Use Case Name	Primary Actor	Description
Manage Users	System Administrator	Allows the administrator to create, update, or deactivate user accounts, assign roles and permissions, and manage system access.
System Backup	System Administrator	Allows the administrator to manually initiate a full database backup, view backup history, and restore data from previous backups.

3.2.4 Relationships

- **Association:** A solid line connecting an actor to a use case indicates that the actor interacts with that functionality.
- **Include Relationship:** An arrowed dashed line labeled <<include>> between two use cases indicates that one use case (the base) *must* include the behavior of another (the included). For example:
    - Process Book Loan <<include>> Validate Member (The system must validate a member before processing a loan).
    - Process Book Return <<include>> Calculate Fine (The system must check for and calculate fines upon every return).
- **Extend Relationship:** An arrowed dashed line labeled <<extend>> indicates that one use case (the extension) may optionally extend the behavior of another under specific conditions. For example:
    - Notify Member <<extend>> Process Book Return (If a returned book was reserved, the system *optionally* extends the process by notifying the member who reserved it).

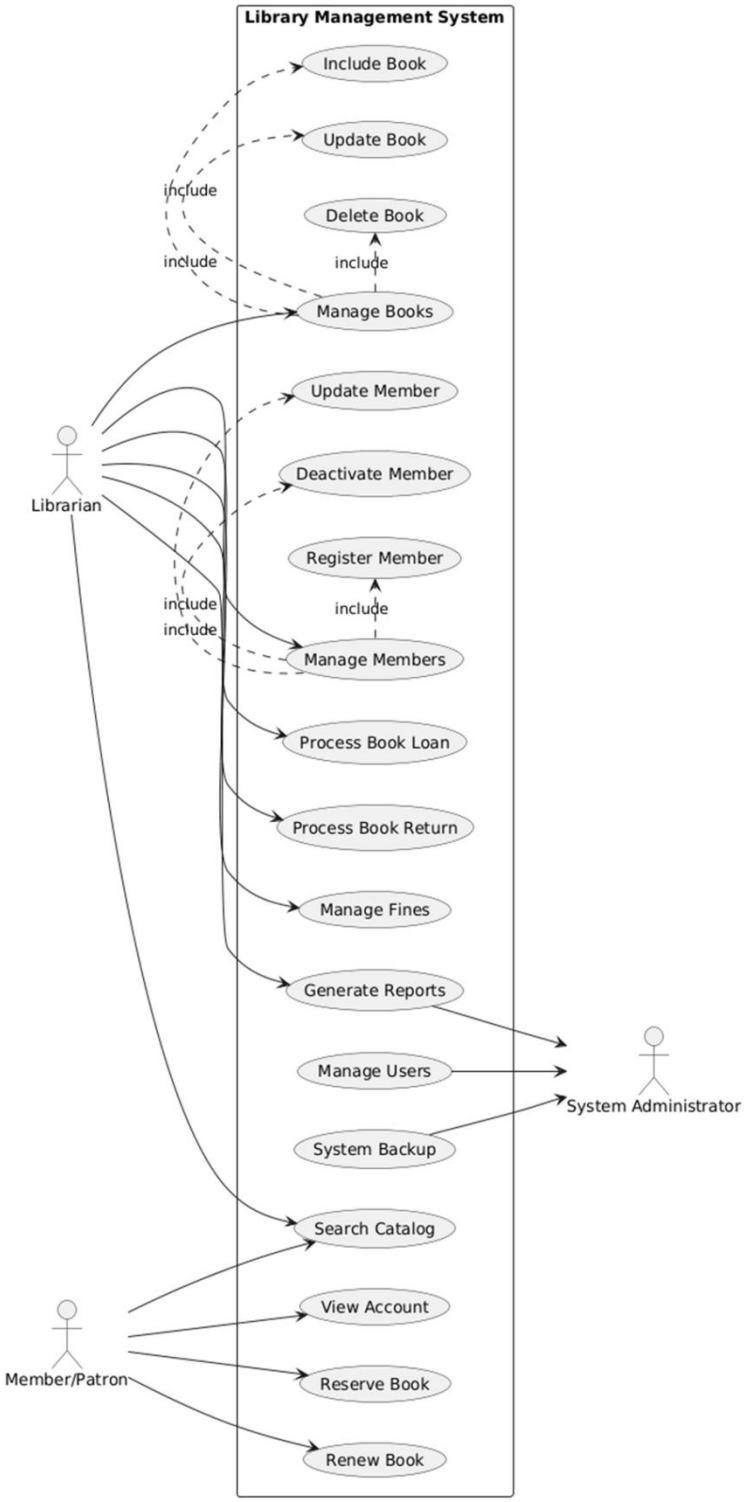
3.2.5 Graphical Representation

The diagram below visually summarizes the actors and their associated use cases, illustrating the system's scope and user interactions.

The diagram shows a system boundary box labeled **"Library Management System"** containing all the use cases. Outside the box, the three actors are placed:

- The **Librarian** on the left, connected to: *Manage Books, Manage Members, Process Book Loan, Process Book Return, Manage Fines, Generate Reports.*
- The **Member/Patron** on the right, connected to: *Search Catalog, View Account, Reserve Book, Renew Book.*
- The **System Administrator** at the bottom, connected to: *Manage Users, System Backup.*

The <<include>> and <<extend>> relationships are shown as dashed arrows between the relevant use cases inside the system boundary.



3.3 Class Diagram

The Class Diagram is a static structure diagram that visualizes the main entities (classes) in the Library Management System, their characteristics (attributes), their

behaviors (operations), and, most importantly, how they relate to one another. It forms the foundation for both the system's database design and its object-oriented structure.

3.3.1 Overview of Classes

The core entities of the Library Management System and their key attributes and operations are defined below.

3.3.2 Detailed Class Descriptions

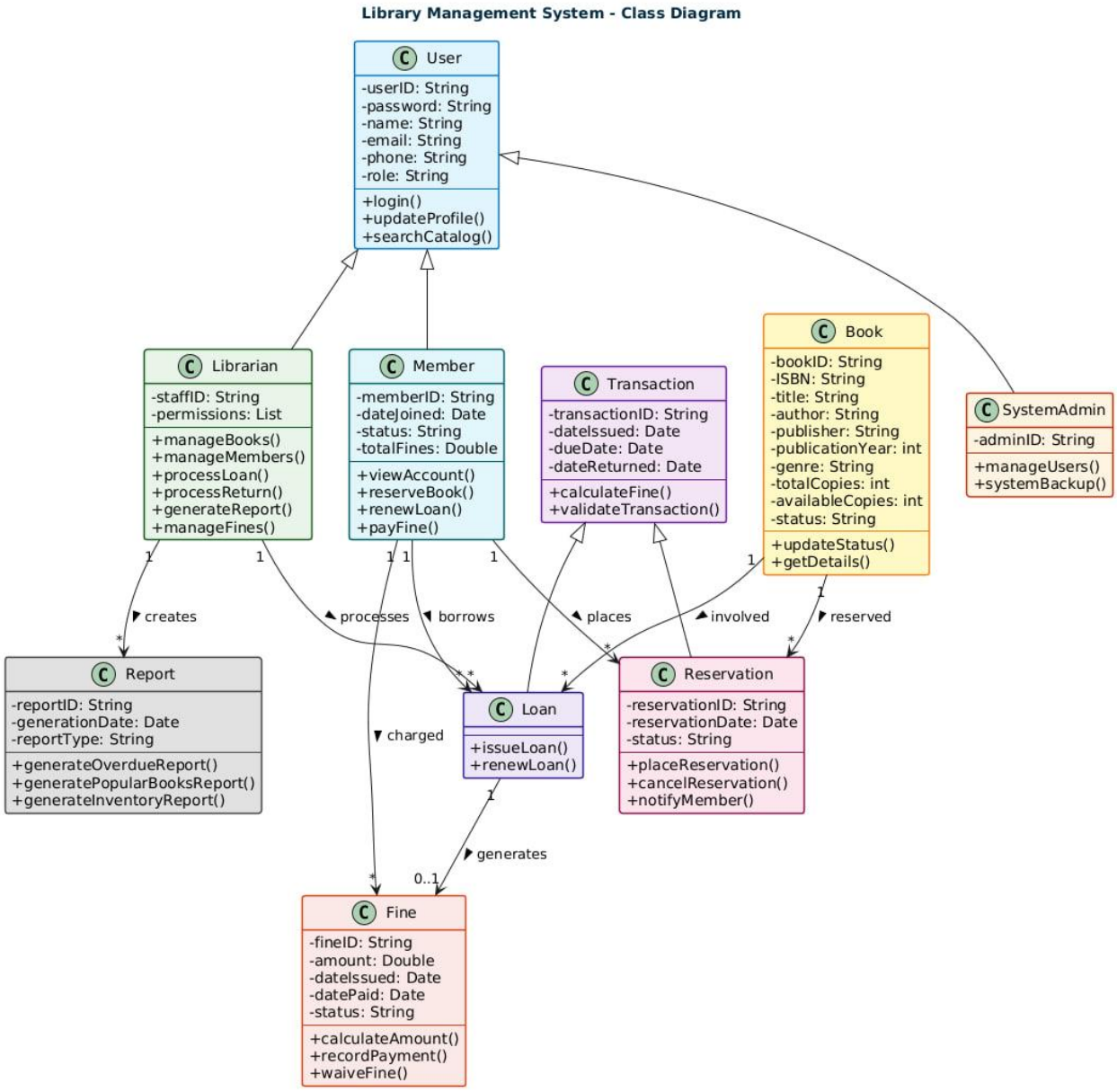
Class Name	Attributes	Methods	Description
User	userID, password, name, email, phone, role	login(), updateProfile(), searchCatalog()	General class for all system users; specialized into Member, Librarian, and SystemAdmin.
Member	memberID, dateJoined, status, totalFines	viewAccount(), reserveBook(), renewLoan(), payFine()	Subclass of User; represents library patrons who borrow books.
Librarian	staffID, permissions	manageBooks(), manageMembers(), processLoan(), processReturn(), generateReport(), manageFines()	Subclass of User; handles daily library operations.
SystemAdmin	adminID	manageUsers(), systemBackup()	Subclass of User; responsible for technical maintenance and user management.
Book	bookID, ISBN, title, author, publisher, publicationYear,	updateStatus(), getDetails()	Represents a book or resource in the library’s collection.

Class Name	Attributes	Methods	Description
	genre, totalCopies, availableCopies, status		
Transaction	transactionID, dateIssued, dueDate, dateReturned	calculateFine(), validateTransaction()	Abstract class generalizing borrowing transactions.
Loan	(inherits from Transaction)	issueLoan(), renewLoan()	Subclass of Transaction; represents borrowing a book.
Reservation	reservationID, reservationDate, status	placeReservation(), cancelReservation(), notifyMember()	Represents a member's hold on a book currently on loan.
Fine	fineID, amount, dateIssued, datePaid, status	calculateAmount(), recordPayment(), waiveFine()	Represents financial penalties for overdue books.
Report	reportID, generationDate, reportType	generateOverdueReport(), generatePopularBooksReport(), generateInventoryReport()	Represents analytical and operational reports generated by the system.

3.3.3 Relationships and Multiplicity

- Inheritance (Generalization):
    - Member, Librarian, and SystemAdmin inherit from User (triangle arrowhead).

    - Loan and Reservation inherit from Transaction.
- Associations:
    - Member 1 --- * Loan: A Member can have many Loans, but a Loan is associated with exactly one Member. (1-to-Many)
    - Book 1 --- * Loan: A Book can be part of many Loans over time, but a specific Loan is for one Book. (1-to-Many)
    - Librarian 1 --- * Loan: A Librarian processes many Loans, but a Loan is processed by one Librarian. (1-to-Many)
    - Member 1 --- * Reservation: A Member can have many Reservations, but a Reservation belongs to one Member. (1-to-Many)
    - Book 1 --- * Reservation: A Book can have many Reservations over time, but a specific Reservation is for one Book. (1-to-Many)
    - Loan 1 --- 0..1 Fine: A Loan may lead to one Fine if it is overdue, or zero Fines if it is returned on time. (1-to-Zero-or-One)
    - Member 1 --- * Fine: A Member can have many Fines. (1-to-Many)



3.4 LMS Sequence Diagram

A sequence diagram is a type of UML diagram that models the interaction between objects or actors over time. It shows how messages are exchanged to accomplish a specific task or use case, emphasizing the order of events.

Actors / Lifelines

- ✓ Librarian (Actor): Performs book management and loan operations.
- ✓ Member (Actor): Performs borrow, reserve, return, and payment actions.
- ✓ LoanInterface, ReservationInterface, ReturnInterface, PaymentInterface, CatalogInterface (Boundary Objects): Entry points for system interactions.
- ✓ LoanController, ReservationController, ReturnController, PaymentController, CatalogController (Control Objects): Contain business logic and validations.
- ✓ Book, Loan, Member (Entity Objects): Represent database entities.

1. Process Book Loan

- Purpose: Handles issuing a book to a member.
- Flow:
    1. Librarian requests Process Loan through LoanInterface.
    2. LoanInterface calls createLoan(memberID, bookID) on LoanController.
    3. LoanController retrieves member details and book details.
    4. LoanController executes validateLoan() business logic.
    5. If validation passes:
        - Creates Loan entity with issue/due dates.
        - Updates Book status and available copies.
        - Updates Member's current loans.
        - Sends confirmation back to Librarian.
    6. If validation fails:
        - Sends validation error back to Librarian.
- Notes:
    - Alternative paths handled using alt.
    - Shows clear chain of communication between actor, boundary, control, and entity objects.

2. Member Reserve Book

- Purpose: Member reserves a book that is currently unavailable.
- Flow:
    1. Member initiates Reserve Book via ReservationInterface.
    2. ReservationInterface calls createReservation(memberID, bookID) on ReservationController.
    3. ReservationController gets book details and validates member eligibility.
    4. Creates reservation record and sends confirmation back to the Member.
- Notes:
    - Linear workflow; no complex alternative paths.
    - Emphasizes interaction between Member and system for reservations.

3. Member Borrow Book

- Purpose: Member borrows an available book.
- Flow:
    1. Member requests Borrow Book through LoanInterface.
    2. LoanInterface calls LoanController.
    3. LoanController retrieves book details and checks borrowing limit.
    4. If the member can borrow:
        - Creates Loan entity, updates Book status, adds loan to Member.
        - Sends confirmation.
    5. If the member cannot borrow (limit reached):
        - Sends limitError back to Member.
- Notes:
    - Uses alt for alternative flows (success vs. failure).

4. Member Return Book

- Purpose: Member returns a borrowed book.

- Flow:
    1. Member initiates Return Book via ReturnInterface.
    2. ReturnController updates Book status to available and marks Loan as returned.
    3. Checks for overdue: if overdue, calculates fine and notifies Member.
    4. Sends return confirmation back to Member.
- Notes:
    - Decision for overdue handled via alt.
    - Illustrates interaction between Member, ReturnController, Book, and Loan.

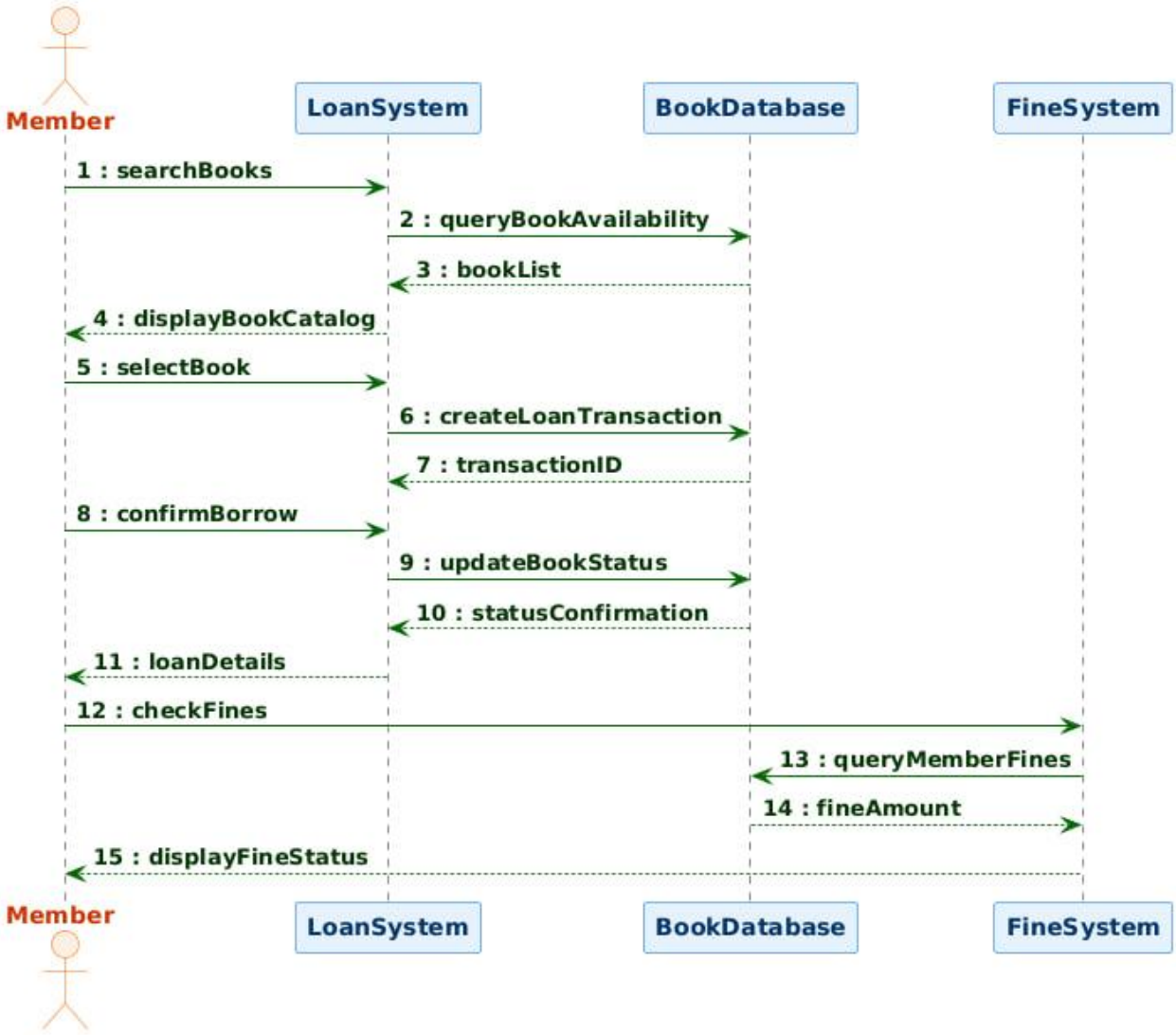
5. Member Pay Fine

- Purpose: Member pays outstanding fines.
- Flow:
    1. Member initiates Pay Fine via PaymentInterface.
    2. PaymentInterface calls PaymentController to processPayment(memberID, amount).
    3. PaymentController validates payment:
        - Successful → updates Member record and sends confirmation.
        - Failed → sends paymentError back to Member.
- Notes:
    - Alternative paths represented using alt.
    - Shows interaction between Member and system for financial transactions.

6. Librarian Add Book

- Purpose: Librarian adds a new book to the catalog.
- Flow:
    1. Librarian requests Add Book via CatalogInterface.

2. CatalogInterface calls CatalogController.
3. Catalog Controller validates book details and saves the book.
4. Sends confirmation back to Librarian.



Notes:

- Linear workflow, no decision paths.
- Demonstrates system validation and persistence for catalog management.

- Sequence diagrams focus on time-ordered interactions.
- Boundary objects act as entry points for actors.
- Control objects handle business logic.

- Entity objects represent the database records or persistent objects.
- Decision points and alternative flows are represented with alt blocks.

This set covers core LMS functionalities: loaning, reserving, borrowing, returning, paying fines, and catalog management.

3.5 Activity Diagram

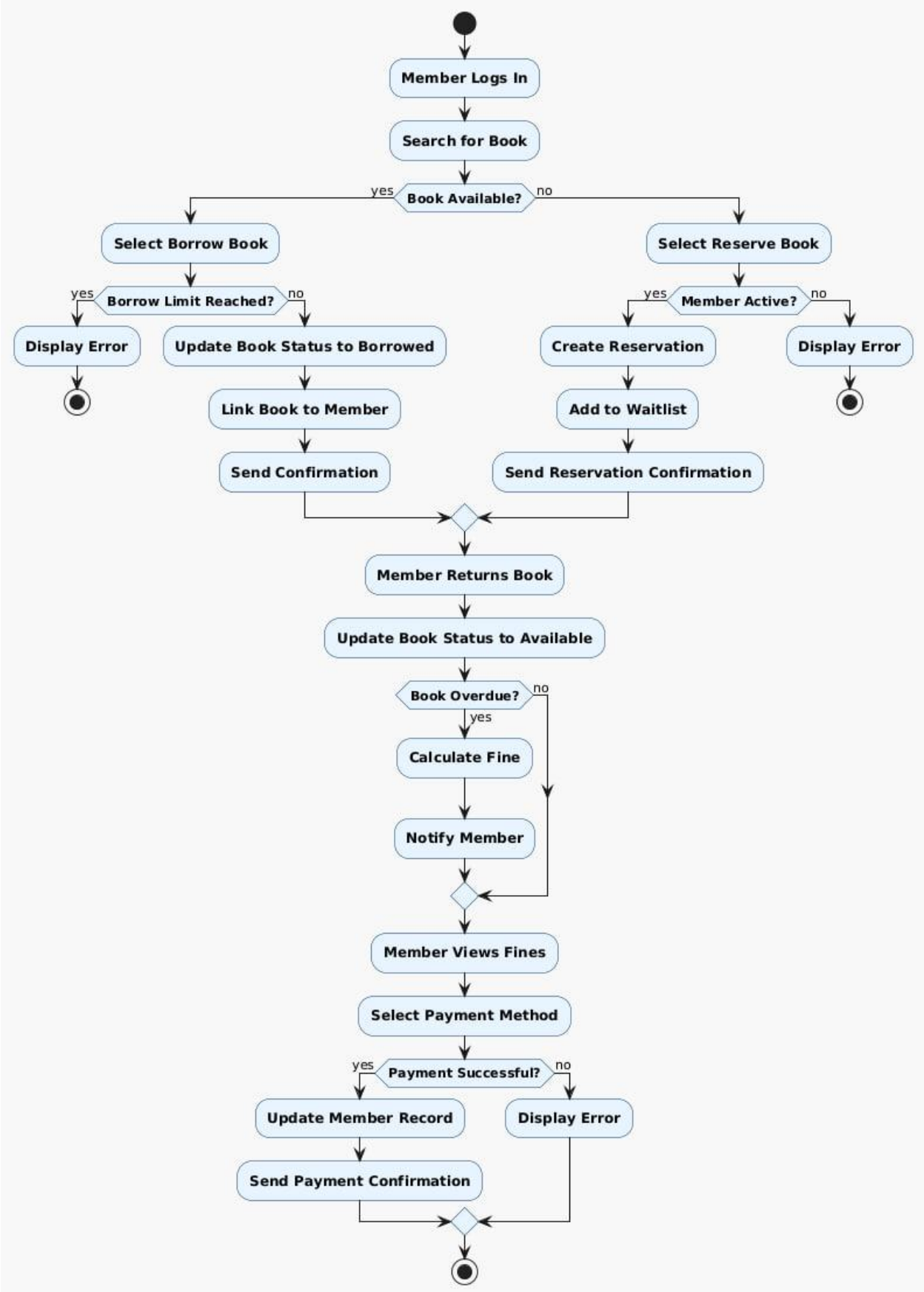
Activity diagrams are used to model dynamic workflows in a system. They show the sequence of actions, decision points, and interactions between actors (Members, Librarians) and the system.

Purpose in LMS:

- Visualize workflows for reserving, borrowing, returning, paying fines, and catalog management.
- Highlight decision points, alternate flows, and error handling.
- Help with design, documentation, and testing.

Actors / Swimlanes:

- Member: Performs actions like login, search, borrow, return, or pay fines.
- System: Performs validations, updates records, sends confirmations, handles errors.
- Librarian: Performs actions like login and managing catalog entries.


Member Logs In
Search for Book
Book Available?
yes
no
Select Borrow Book
Select Reserve Book
Borrow Limit Reached?
yes
no
Member Active?
yes
no
Display Error
Update Book Status to Borrowed
Create Reservation
Display Error
Link Book to Member
Add to Waitlist
Send Confirmation
Send Reservation Confirmation
Member Returns Book
Update Book Status to Available
Book Overdue?
no
yes
Calculate Fine
Notify Member
Member Views Fines
Select Payment Method
Payment Successful?
yes
no
Update Member Record
Display Error
Send Payment Confirmation

Workflow Descriptions

1. Member Reserving a Book

- Start Node: Member initiates workflow.
- Member logs in and searches for a book.
- Selects “Reserve” for a Checked-Out book.
- System verifies member status:
    - Inactive: Display error → end workflow.
    - Active: Create reservation, link to member and book.
- If the book is unavailable, system adds member to waitlist.
- System sends reservation confirmation.

Decision nodes: Member active? Book available?
End Node: Reservation complete.

2. Member Borrowing a Book

- Member logs in, searches, and selects Borrow Book.
- System checks if the book is available:
    - Not available → display error → end.
- System checks borrowing limit:
    - Limit reached → display error → end.
    - Otherwise → update book status to borrowed, link to member, send confirmation.

Decision nodes: Book available? Borrow limit reached?

3. Member Returning a Book

- Member logs in and selects the book to return.
- System updates book status to Available.
- System checks if the book is overdue:

    - If yes → calculate fine and notify member.
- System sends return confirmation.

Decision node: Book overdue?

4. Member Paying Fines

- Member logs in, views outstanding fines, selects payment method.
- System processes payment:
    - Failed → display error → end.
    - Successful → update member record and send confirmation.

Decision node: Payment successful?

5. Librarian Adding a New Book

- Librarian logs in and enters new book details.
- System validates and saves the book in catalog.
- System sends confirmation of addition.

Notes:

- Linear workflow with no decision nodes.
- Shows actor-system interaction for catalog management.

6. Librarian Updating Book Information

- Librarian logs in and searches for the book to edit.
- System validates changes and saves them.
- System optionally notifies members if needed.
- **Decision nodes:** None (linear workflow).
- **Swimlanes:** Librarian, System.

7. Librarian Managing Reservations / Waitlist

- Librarian logs in and views pending reservations.
- System asks for approval decision:

    - Approve → confirm reservation, notify member.
    - Reject → cancel reservation, notify member.
- **Decision node:** Reservation approve?
- **Swimlanes:** Librarian, System.

8. Member Registering for an Account

- Member enters registration details.
- System verifies information:
    - Invalid → display error → end workflow.
    - Valid → create account and send welcome email.
- **Decision node:** Information valid?
- **Swimlanes:** Member, System.

9. System Notifications (Overdue / Reservation Availability)

- System checks for overdue books:
    - If exists → notify members.
- System checks pending reservations for available books:
    - If available → notify members.
- **Decision nodes:** Overdue exists? Reservations available?

Chapter 4: System Design

4.1 Overview

System design is the phase where the analytical models from Chapter 3 are transformed into a blueprint for implementation. This chapter outlines the high-level and low-level design elements of the Library Management System (LMS), ensuring that the system is scalable, maintainable, secure, and aligned with the requirements elicited and analyzed earlier. The overview provides a comprehensive introduction to the design principles, strategies, and components that form the foundation of the LMS.

The design process for the LMS adopts a modular, object-oriented approach, leveraging best practices from software engineering standards. It focuses on creating a system that integrates seamlessly with library operations, supporting functionalities like catalog management, member services, circulation, fine management, and reporting.

Key design goals include:

- Modularity: Breaking the system into independent modules (e.g., User Management, Book Management) to facilitate development, testing, and maintenance.
- Scalability: Allowing the system to handle a 50% growth in books, members, and transactions, as specified in the non-functional requirements.
- Security: Implementing role-based access control, data encryption, and audit logging to protect sensitive member and transaction data.
- Usability: Designing intuitive, responsive web interfaces for diverse users (librarians, members, system administrators).
- Performance: Ensuring critical transactions (e.g., checkout, return) are processed in under 1 second and search operations return results in under 2 seconds.
- Reliability: Incorporating database transaction management and automated backups to ensure data integrity and availability.

The design is iterative, refined through the Agile sprints outlined in the methodology. Technologies selected include:

- ✓ Frontend: HTML5, CSS3, JavaScript for a responsive and accessible user interface.
- ✓ Backend: PHP with a framework like Laravel or Node.js with Express.js for robust server-side logic and RESTful API development.
- ✓ Database: MySQL or PostgreSQL for reliable, ACID-compliant relational data storage.
- ✓ Deployment: Hosting on a university server or cloud platform like AWS, with Git for version control.

This overview also considers design patterns: MVC (Model-View-Controller) for a clear separation of concerns, Singleton for managing database connections, and Factory for creating complex objects like different types of reports.

Risks addressed include data migration from legacy manual systems, handled via dedicated data import utilities, and user acceptance, mitigated through the phased rollout and training as per the feasibility study.

In summary, this chapter's design ensures the LMS is not only functional but also a robust, future-proof platform capable of evolving with the library's needs.

4.1.1 Design Principles and Standards

The system adheres to fundamental software design principles:

- ➢ High Cohesion and Low Coupling: Each module (e.g., FineManagement, ReportGenerator) has a single, well-defined purpose and interacts with others through well-defined interfaces, minimizing dependencies.
- ➢ SOLID Principles: The object-oriented design follows these principles to ensure the code is easy to manage and extend. For example, the Single Responsibility Principle is reflected in classes like Book (managing book data) and Fine (managing fine calculations).
- ➢ Standards: UML 2.5 is used for all visual models. Security practices follow OWASP guidelines to prevent common vulnerabilities like SQL injection and cross-site scripting (XSS).

4.1.2 Design Process Workflow

The design process follows a structured workflow:

- ✓ Requirement Mapping: Each functional requirement from Chapter 2 is mapped to one or more system components.
- ✓ Architectural Design: Defining the high-level 3-tier structure and communication protocols.
- ✓ Component Detailing: Elaborating each component's interfaces, as shown in the Component Diagram.
- ✓ Data Modeling: Transforming the conceptual Class Diagram into a normalized physical database schema.
- ✓ Validation: Reviewing all design artifacts against the original requirements to ensure consistency and completeness.

4.1.3 High-Level Design Choices

A key design choice was the selection of a 3-Tier Monolithic Architecture over a Microservices architecture. While microservices offer benefits, the initial scale of the LMS, its tight integration of core functions, and the development team's size make a well-structured monolithic application the most efficient and manageable choice. This allows for simpler deployment, testing, and transaction management while still maintaining modularity within the codebase.

4.1.4 Integration with Previous Chapters

This design chapter is a direct translation of the analysis from Chapter 3. The Class Diagram's entities become the Persistent Data Model. The Use Cases from the Use Case Diagram define the operations of the components in the Component Diagram. The Non-Functional Requirements from Chapter 2 (e.g., performance, security) directly inform the architectural decisions and technology selections described here.

4.2 System Architecture

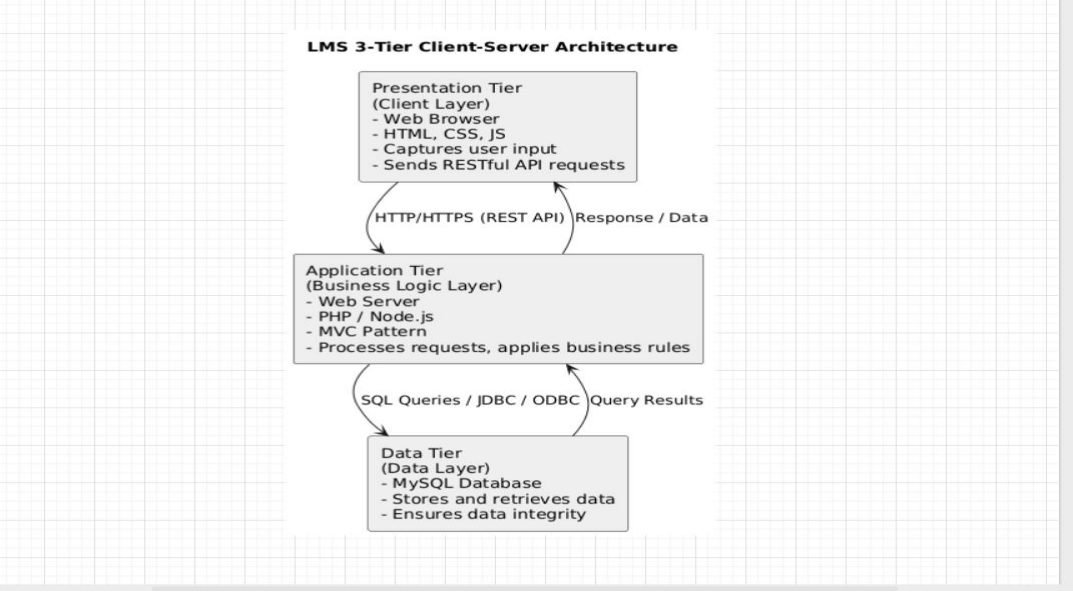
The system architecture describes the overall structure of the LMS, including layers, components, and interactions. It adopts a 3-Tier Layered Architecture to promote separation of concerns, enhancing maintainability, scalability, and security.

4.2.1 Architectural Style

The LMS uses a 3-Tier Client-Server Architecture, consisting of:

- Presentation Tier (Client Layer): Runs in the user's web browser. It is responsible for rendering the user interface and capturing user input, built with HTML, CSS, and JavaScript. It communicates with the Application Tier via HTTP/HTTPS requests (RESTful API).
- Application Tier (Business Logic Layer): Hosted on a web server, this tier contains the core business logic of the LMS. It processes requests from the Presentation Tier, executes operations (e.g., validating a loan, calculating a fine), and interacts with the Data Tier. It is built using PHP/Node.js and follows the MVC pattern.
- Data Tier (Data Layer): Consists of a relational database management system (RDBMS) like MySQL. It is solely responsible for data storage, retrieval, and integrity.

This tiered architecture allows each layer to be developed, scaled, and maintained independently.



4.2.2 Key Architectural Components

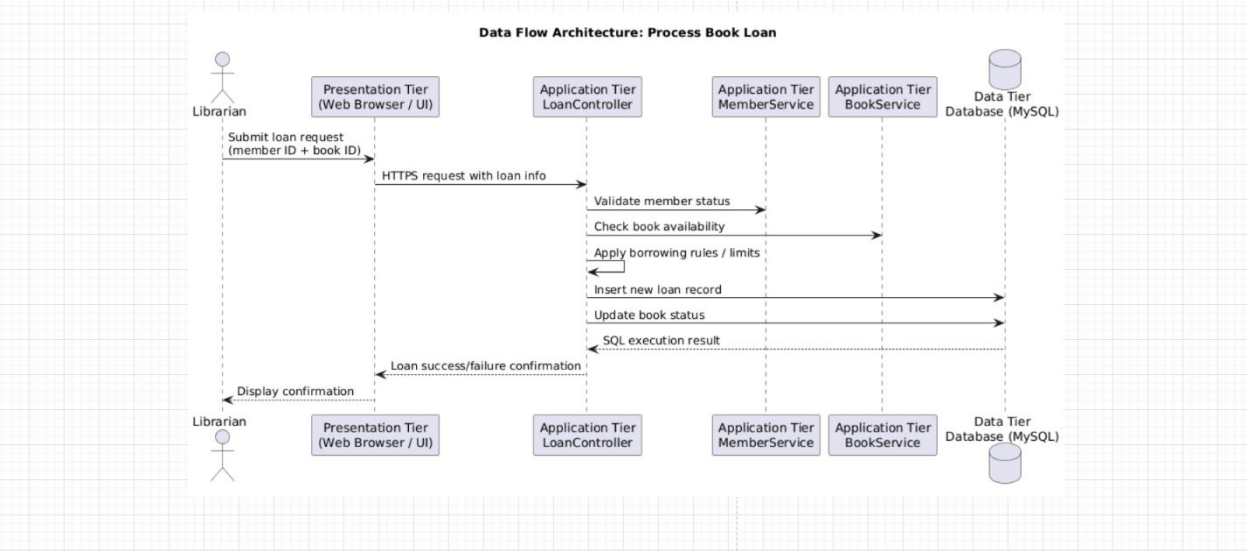
➢ Presentation Layer (Frontend): Comprises web pages and forms for all user interactions. It includes dedicated interfaces for Librarians (admin dashboard), Members (personal dashboard), and System Administrators.

➢ Application Layer (Backend): Contains the controllers and services that implement the system's functionality.
- UserController: Handles authentication, registration, and profile management.
- BookController: Manages all CRUD operations for the book catalog.
- LoanController: Processes book check-out, check-in, and renewal.
- FineController: Manages fine calculation, payment recording, and waivers.
- ReportController: Generates and serves various reports.
➢ Data Layer (Database): The MySQL database stores all persistent data. The schema is designed based on the data model in section 4.5.
➢ Security Layer (Cross-Cutting): Integrated across all tiers. It includes password hashing in the database, role-based access control in the application logic, and HTTPS enforcement for data in transit.

4.2.3 Data Flow Architecture

A typical data flow for the "Process Book Loan" use case is as follows:

1. Librarian submits a loan request via the Presentation Tier (e.g., scans member and book IDs).
2. The request is sent via HTTPS to the LoanController in the Application Tier.
3. The LoanController calls the MemberService and BookService to validate the member's status and the book's availability.
4. The Application Tier executes business logic (e.g., applies borrowing limits).
5. If valid, the LoanController sends SQL commands to the Data Tier to create a new loan record and update the book's status.
6. A success confirmation flows back through the tiers to the Librarian.



4.2.4 Scalability and Performance Strategies

- Horizontal Scaling: The Application Tier can be scaled by adding more web server instances behind a load balancer.
- Caching: Frequently accessed, read-heavy data (e.g., book catalog for searches) can be cached using Redis or Memcached to reduce database load and improve response times.
- Database Optimization: Strategic use of indexes on frequently queried columns (e.g., book_id, title, member_id) ensures that performance remains within the required 2-second threshold for searches.

4.2.5 Security Architecture

Security is implemented in depth:

- Network Level: Firewalls restrict access to the server.

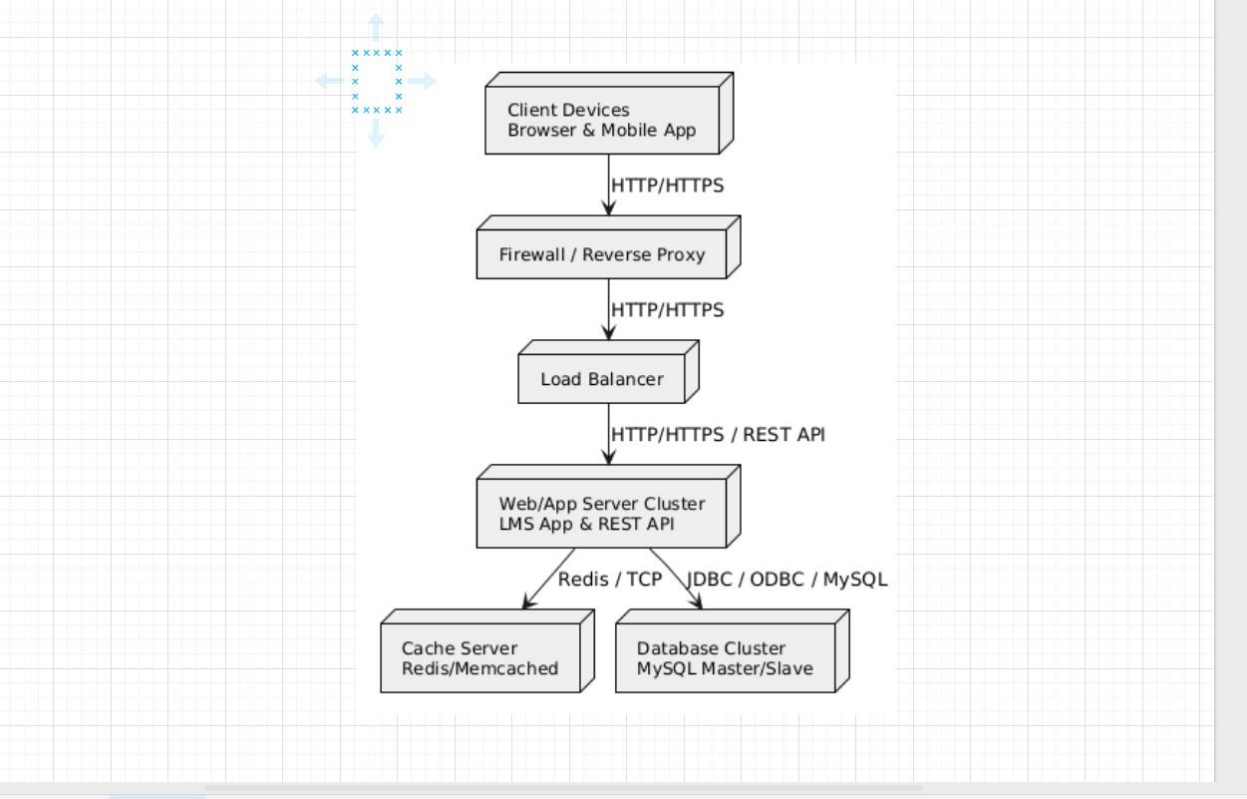
- Application Level: Input validation and sanitization prevent injection attacks. Session management with timeouts protects against hijacking.
- Data Level: Passwords are stored as salted hashes. Sensitive data can be encrypted at rest.

4.2.6 Fault Tolerance and Recovery

- Redundancy: Database replication can be set up for failover.
- Backups: Automated daily backups (as per non-functional requirements) ensure data can be recovered in case of failure.
- Transactions: Database transactions are used for operations like loan processing to ensure data remains consistent even in case of errors.

4.3 Deployment Diagram

The deployment diagram illustrates the physical architecture of the Library Management System (LMS), showing how software components are deployed across hardware nodes. The system is designed as a web-based application deployable on a university server or a cloud provider.



4.3.1 Diagram Description

The deployment diagram illustrates the **physical architecture** of the Library Management System (LMS), showing how software components are deployed across hardware nodes. The system is designed as a web-based application, deployable on a university server, cloud provider, or local infrastructure.

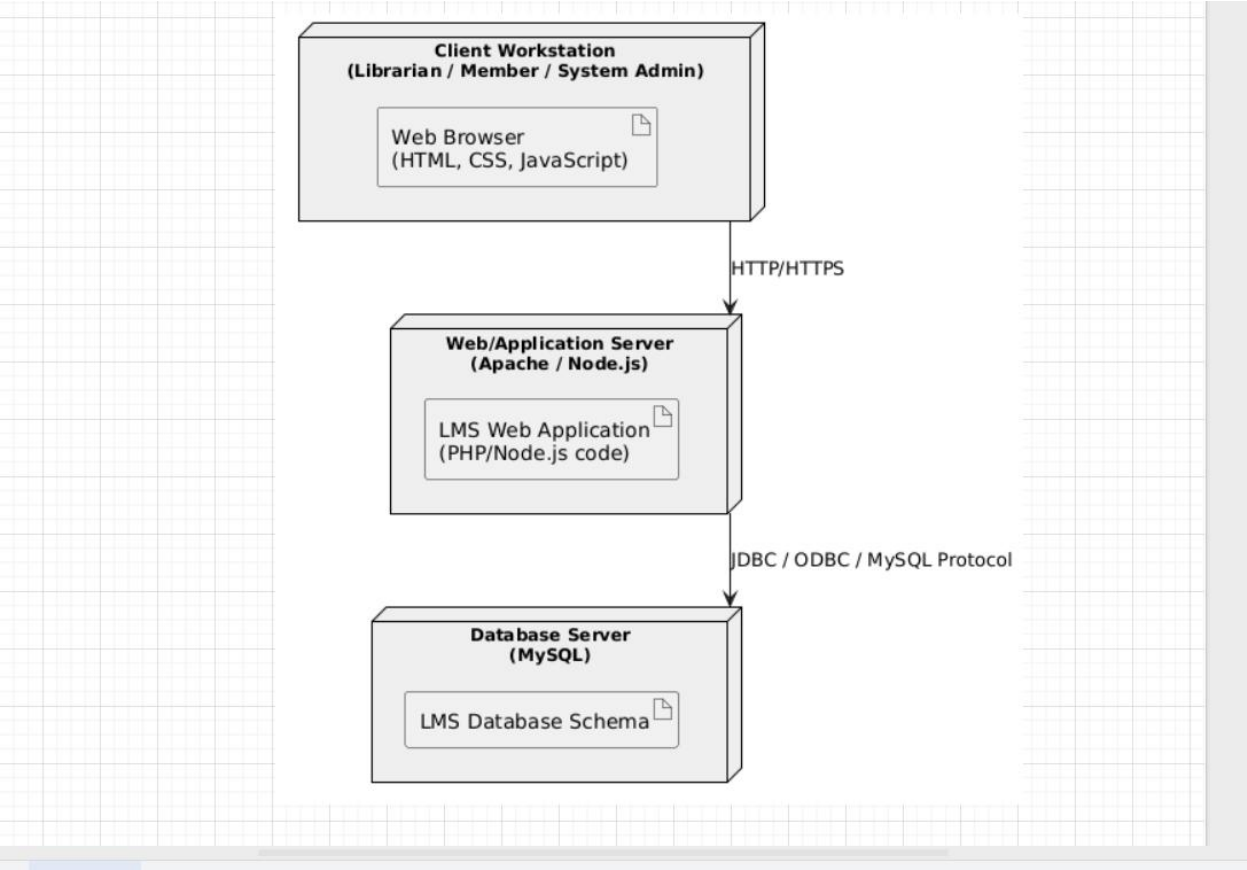
Nodes and Artifacts (from the diagram):

1. **Client Devices:**
   Represents the devices used by Librarians, Members, and System Admins. The artifacts deployed here are:
   - **Web Browser:** Renders the LMS web interface (HTML, CSS, JavaScript)
   - **Mobile App:** Provides mobile access via REST API
2. **Firewall / Reverse Proxy:**
   Protects the web/application servers from unauthorized access and manages HTTPS/SSL traffic.
3. **Load Balancer:**
   Distributes client requests across multiple web/application servers to ensure high availability and scalability.
4. **Web/Application Server Cluster:**
   Hosts the LMS Web Application and REST API. Artifacts deployed here include:
   - **LMS Web Application** (PHP or Node.js code)
   - **REST API** for mobile and external client access
5. **Cache Server:**
   Optional server (e.g., Redis or Memcached) to store frequently accessed data for improved performance.
6. **Database Server Cluster:**
   Hosts the LMS database schema in MySQL. The cluster may include master/slave or replication setup for reliability and high availability.

Connections and Communication Protocols (as shown in the diagram):

- **Client Devices → Firewall / Reverse Proxy:** HTTP/HTTPS
- **Firewall → Load Balancer:** HTTP/HTTPS
- **Load Balancer → Web/Application Server Cluster:** HTTP/HTTPS or REST API
- **Web/Application Server Cluster → Cache Server:** Redis / TCP
- **Web/Application Server Cluster → Database Server Cluster:** JDBC / ODBC / MySQL native protocol

This architecture ensures the LMS system is **secure, scalable, performant, and highly available**, matching the physical deployment represented in the diagram.



4.3.2 Nodes and Artifacts

➢ Client Nodes: Standard PCs or laptops; artifact: Web Browser.
➢ Web Server Node: A virtual machine or physical server; artifacts: LMS Web Application (PHP/JS files), Web Server (Apache/Nginx).
➢ Database Server Node: A dedicated machine for database operations; artifact: LMS Database Schema (MySQL/PostgreSQL).

4.3.3 Deployment Strategies

A simple, reliable deployment strategy is recommended. The application can be deployed directly to the university server. Version control with Git allows for easy rollbacks if necessary.

4.3.4 Hardware Requirements

- Web/Application Server: Minimum 2 vCPU, 4GB RAM.
- Database Server: Minimum 2 vCPU, 4GB RAM, with SSD storage for better I/O performance.
- These requirements are minimal and align with the feasibility study's assessment of using existing UoG infrastructure.

4.3.5 Cloud vs. On-Premise

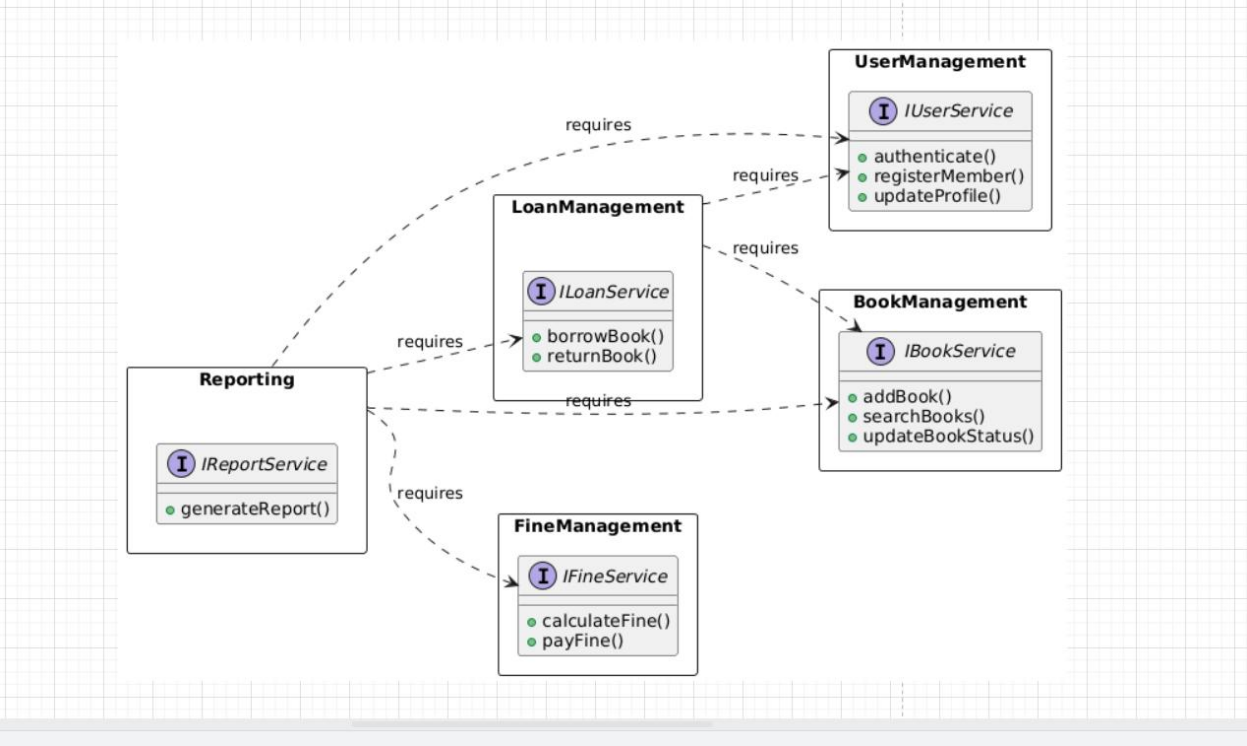
The initial design is for an On-Premise deployment on the university's server, as stated in the feasibility study. However, the 3-tier architecture makes it inherently portable to a Cloud environment (e.g., AWS, Azure) in the future if needed for greater scalability or accessibility.

4.4 Component Diagram

The Component Diagram shows the organization of the software components within the Application Tier and their dependencies, focusing on reusable and replaceable parts.

4.4.1 Diagram Description

The diagram depicts major components as rectangles with lollipop symbols representing provided interfaces and socket symbols representing required interfaces.



4.4.2 Key Components

✓ UserManagement Component
✓ Provides IUserService interface

Methods: authenticate(), registerMember(), updateProfile()

✓ BookManagement Component
✓ Provides IBookService interface

Methods: addBook(), searchBooks(), updateBookStatus()

✓ LoanManagement Component
✓ Provides ILoanService interface
✓ Requires IUserService and IBookService interfaces
✓ Orchestrates borrowing and returning of books
✓ FineManagement Component
✓ Provides IFineService interface
✓ Handles calculation and management of overdue fines

- ✓ Reporting Component
- ✓ Provides IReportService interface
- ✓ Requires interfaces from other components to gather data
- ✓ Generates various reports

4.4.3 Interfaces and Dependencies

Dependencies are managed through these well-defined interfaces. For example, the LoanManagement component depends on the IUserService and IBookService interfaces, not their concrete implementations. This allows for easier testing (e.g., using mocks) and future modifications.

4.4.4 Component Reuse

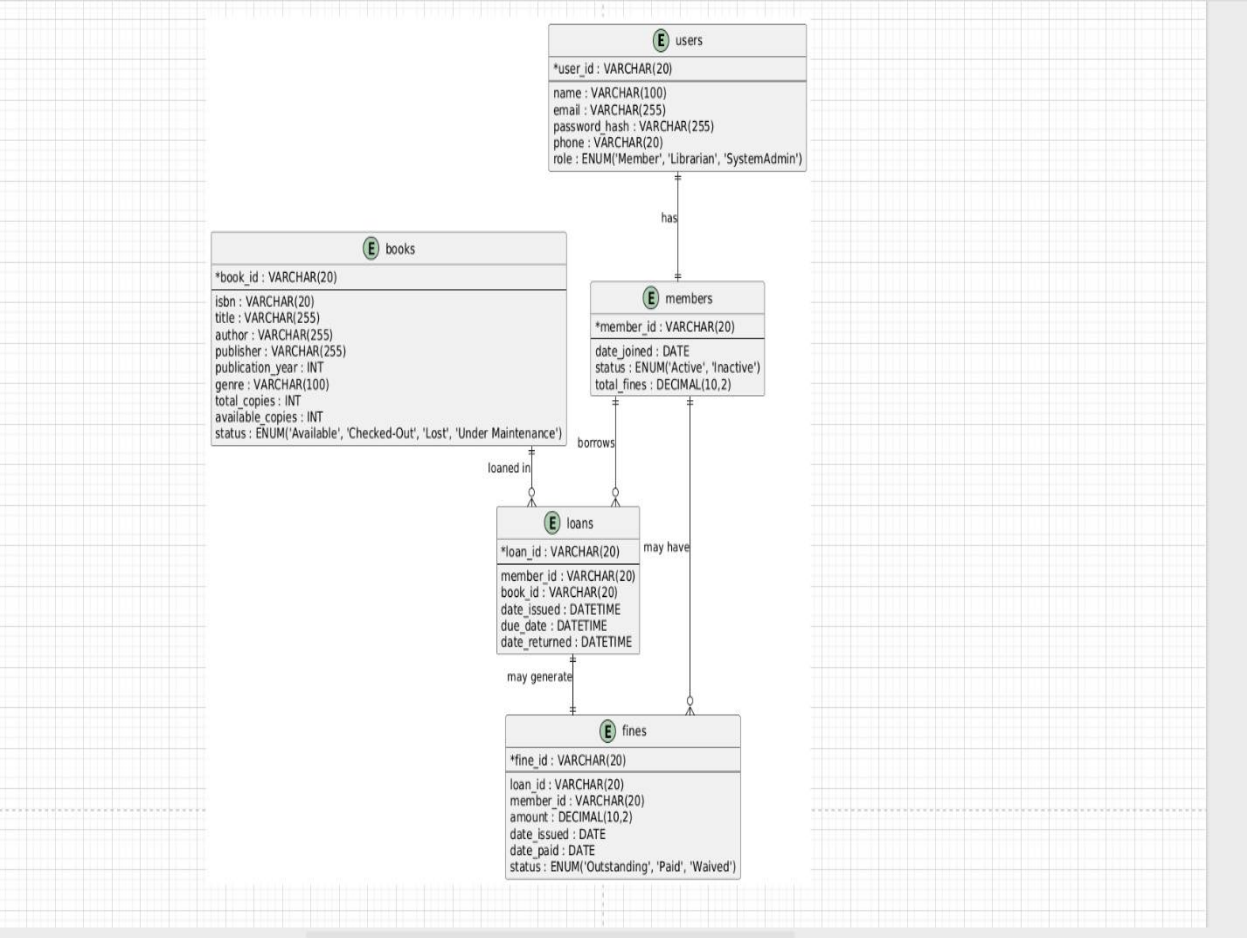
The modular design allows for components to be reused. For instance, the BookManagement component and its IBookService interface could be reused in a future module for managing digital resources or a public OPAC (Online Public Access Catalog).

4.5 Persistent Data Model

The Persistent Data Model defines how data is structured, stored, and accessed in the relational database. It is a direct implementation of the Class Diagram from the system analysis, optimized for performance and integrity.

4.5.1 ER Diagram

The Entity-Relationship Diagram would visually represent the tables listed below and their relationships (e.g., Member has a one-to-many relationship with Loan).



4.5.2 Relational Schema

The database consists of the following core tables, normalized to the Third Normal Form (3NF) to minimize redundancy:

➢ users Table
- user_id VARCHAR(20) PRIMARY KEY
- name VARCHAR(100) NOT NULL
- email VARCHAR(255) UNIQUE NOT NULL
- password_hash VARCHAR(255) NOT NULL
- phone VARCHAR(20)
- role ENUM('Member', 'Librarian', 'SystemAdmin') NOT NULL

➢ members Table
- member_id VARCHAR(20) PRIMARY KEY, FOREIGN KEY REFERENCES users(user_id)
- date_joined DATE NOT NULL

- status ENUM('Active', 'Inactive') DEFAULT 'Active'
- total_fines DECIMAL(10, 2) DEFAULT 0.00

➢ books Table
- book_id VARCHAR(20) PRIMARY KEY
- isbn VARCHAR(20) UNIQUE
- title VARCHAR(255) NOT NULL
- author VARCHAR(255) NOT NULL
- publisher VARCHAR(255)
- publication_year INT
- genre VARCHAR(100)
- total_copies INT NOT NULL
- available_copies INT NOT NULL
- status ENUM('Available', 'Checked-Out', 'Lost', 'Under Maintenance') DEFAULT 'Available'

➢ loans Table
- loan_id VARCHAR(20) PRIMARY KEY
- member_id VARCHAR(20) FOREIGN KEY REFERENCES members(member_id)
- book_id VARCHAR(20) FOREIGN KEY REFERENCES books(book_id)
- date_issued DATETIME NOT NULL
- due_date DATETIME NOT NULL
- date_returned DATETIME NULL

➢ fines Table
- fine_id VARCHAR(20) PRIMARY KEY
- loan_id VARCHAR(20) UNIQUE FOREIGN KEY REFERENCES loans(loan_id)
- member_id VARCHAR(20) FOREIGN KEY REFERENCES members(member_id)
- amount DECIMAL(10, 2) NOT NULL
- date_issued DATE NOT NULL
- date_paid DATE NULL
- status ENUM('Outstanding', 'Paid', 'Waived') DEFAULT 'Outstanding'

4.5.3 Data Access Layer

An Object-Relational Mapping (ORM) library such as Eloquent (for PHP/Laravel) or Sequelize (for Node.js) will be used to map these database tables to application-level objects (Models). This abstracts raw SQL and simplifies database interactions.

4.5.4 Data Integrity and Backup

- Integrity: Foreign key constraints ensure relational integrity. Check constraints can enforce business rules (e.g., due_date > date_issued).
- Backup: As per the non-functional requirements, automated daily backups will be implemented, with scripts to perform full database dumps.

4.5.5 Performance Optimization

- Indexes: Key fields like book_id, member_id, title, author, and due_date will be indexed to speed up search and lookup operations.

Archiving: As the loans table grows, older, completed transactions can be archived to a separate table to maintain performance for active operations.